

10.9.3 Implementation of Narrowband Lowpass Filters

In Section 10.6 we demonstrated that a multistage implementation of **sampling-rate** conversion often provides for a more efficient realization, especially when the filter specifications are very tight (e.g., a narrow **passband** and a narrow transition band). Under similar conditions, a **lowpass**, linear-phase FIR filter may be more efficiently implemented in a multistage decimator-interpolator configuration. To be more specific, we can employ a multistage implementation of a decimator of size D , followed by a multistage implementation of an interpolator of size I , where $I = D$.

We demonstrate the procedure by means of an example for the design of a **lowpass** filter which has the same specifications as the filter that is given in Example 10.6.1.

Example 10.9.1

Design a linear-phase FIR filter that satisfies the following specifications:

Sampling frequency:	8000 Hz
Passband:	$0 \leq F \leq 75$
Transition band:	$75 \leq F \leq 80$
Stopband	$80 \leq F \leq 4000$
Passband ripple:	$\delta_1 = 10^{-2}$
Stopband ripple:	$\delta_2 = 10^{-4}$

Solution If this filter were designed as a single-rate linear-phase FIR filter, the length of the filter required to meet the specifications is (from Kaiser's formula)

$$M \approx 5152$$

Now, suppose that we employ a **multirate** implementation of the **lowpass** filter based on a decimation and interpolation factor of $D = I = 100$. A single-stage implementation of the decimator-interpolator requires an FIR filter of length

$$\hat{M}_1 \approx \frac{-10 \log_{10}(\delta_1 \delta_2 / 2) - 13}{14.6 \Delta f} + 1 \approx 5480$$

However, there is a significant savings in computational complexity by implementing the decimator and interpolator filters using their corresponding polyphase filters. If we employ linear-phase (symmetric) decimation and interpolation filters, the use of polyphase filters reduces the multiplication rate by a factor of 100.

A significantly more efficient implementation is obtained by using two stages of decimation followed by two stages of interpolation. For example, suppose that we select $D_1 = 50$, $D_2 = 2$, $I_1 = 2$, and $I_2 = 50$. Then the required filter lengths are

$$\hat{M}_1 = \frac{-10 \log_{10}(\delta_1 \delta_2 / 4) - 13}{14.6 \Delta f} + 1 \approx 177$$

$$\hat{M}_2 = \frac{-10 \log_{10}(\delta_1 \delta_2 / 4) - 13}{14.6 \Delta f} + 1 \approx 233$$

Thus we obtain a reduction in the overall filter length of $2(5480)/(177+233) \approx 13.36$. In addition, we obtain further reduction in the multiplication rate by using polyphase

filters. For the first stage of decimation, the reduction in multiplication rate is 50, while for the second stage the reduction in multiplication rate is 100. Further reductions can be obtained by increasing the number of stages of decimation and interpolation.

10.9.4 Implementation of Digital Filter Banks

Filter banks are generally categorized as two types, *analysis* filter banks and *synthesis* filter banks. An analysis filter bank consists of a set of filters, with system functions $\{H_k(z)\}$, arranged in a parallel bank as illustrated in Fig. 10.32a. The frequency response characteristics of this filter bank splits the signal into a corresponding number of subbands. On the other hand, a synthesis filter bank consists of a set of filters with system functions $\{G_k(z)\}$, arranged as shown in Fig. 10.32b, with corresponding inputs $\{y_k(n)\}$. The outputs of the filters are summed to form the synthesized signal $\{x(n)\}$.

Filter banks are often used for performing spectrum analysis and signal synthesis. When a filter bank is employed in the computation of the discrete Fourier

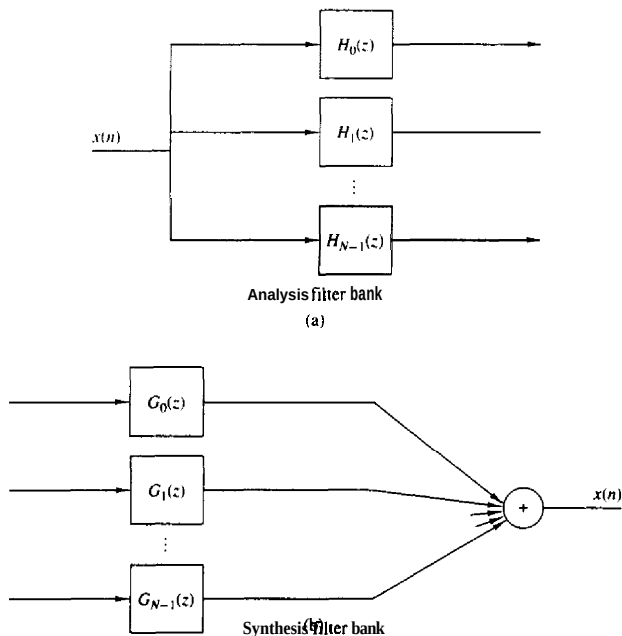


Figure 10.32 A digital filter bank.

transform (DFT) of a sequence $\{x(n)\}$, the filter bank is called a DFT filter bank. An analysis filter bank consisting of N filters $\{H_k(z), k = 0, 1, \dots, N-1\}$ is called a uniform DFT filter bank if $H_k(z), k = 1, 2, \dots, N-1$, are derived from a prototype filter $H_0(z)$, where

$$H_k(\omega) = H_0\left(\omega - \frac{2\pi k}{N}\right) \quad k = 1, 2, \dots, N-1 \quad (10.9.2)$$

Hence the frequency response characteristics of the filters $\{H_k(z), k = 0, 1, \dots, N-1\}$ are simply obtained by **uniformly** shifting the frequency response of the prototype filter by multiples of $2\pi/N$. In the time domain the filters are characterized by their impulse responses, which can be expressed as

$$h_k(n) = h_0(n)e^{j2\pi nk/N} \quad k = 0, 1, \dots, N-1 \quad (10.9.3)$$

where $\{h_0(n)\}$ is the impulse response of the prototype filter.

The uniform DFT analysis filter bank can be realized as shown in Fig. 10.33a, where the frequency components in the sequence $\{x(n)\}$ are translated in frequency to **lowpass** by multiplying $x(n)$ with the complex exponentials $\exp(-j2\pi nk/N)$. $k = 1, \dots, N-1$, and the resulting product signals are passed through a **lowpass** filter with impulse response $\{h_0(n)\}$. Since the output of the **lowpass** filter is relatively narrow in bandwidth, the signal can be decimated by a factor $D \leq N$. The resulting decimated output signal can be expressed as

$$X_k(m) = \sum_n h_0(mD - n)x(n)e^{-j2\pi nk/N} \quad k = 0, 1, \dots, N-1$$

$$m = 0, 1, \dots \quad (10.9.4)$$

where $\{X_k(m)\}$ are samples of the DFT at frequencies $\omega_k = 2\pi k/N$.

The corresponding synthesis filter for each element in the filter bank can be viewed as shown in Fig. 10.33b, where the input signal sequences $\{Y_k(m), k = 0, 1, \dots, N-1\}$ are upsampled by a factor of $I = D$, filtered to remove the images, and translated in frequency by multiplication by the complex exponentials $\{\exp(j2\pi nk/N), k = 0, 1, \dots, N-1\}$. The resulting frequency-translated signals from the N filters are then summed. Thus we obtain the sequence

$$v(n) = \frac{1}{N} \sum_{k=0}^{N-1} e^{j2\pi nk/N} \left[\sum_m Y_k(m) g_0(n - mI) \right]$$

$$= \sum_m g_0(n - mI) \left[\frac{1}{N} \sum_{k=0}^{N-1} Y_k(m) e^{j2\pi nk/N} \right] \quad (10.9.5)$$

$$= \sum_m g_0(n - mI) y_n(m)$$

where the factor $1/N$ is a **normalization** factor, $\{y_n(m)\}$ represent samples of the inverse DFT sequence corresponding to $\{Y_k(m)\}$, $\{g_0(n)\}$ is the impulse response of the **interpolation** filter, and $I = D$.

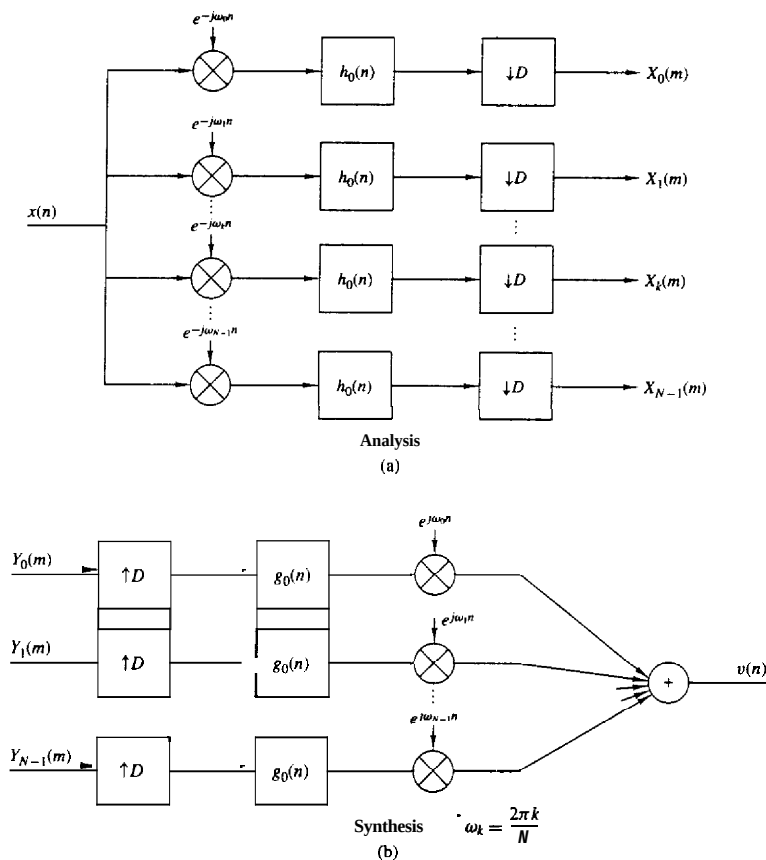


Figure 10.33 A uniform DFT filter bank.

The relationship between the output $\{X_k(n)\}$ of the analysis filter bank and the input $\{Y_k(m)\}$ to the synthesis filter bank depends on the application. Usually, $\{Y_k(m)\}$ is a modified version of $\{X_k(m)\}$, where the specific modification is determined by the application.

An alternative realization of the analysis and synthesis filter banks is illustrated in Fig. 10.34. The filters are realized as **bandpass** filters with impulse responses

$$h_k(n) = h_0(n)e^{j2\pi nk/N} \quad k = 0, 1, \dots, N-1 \quad (10.9.6)$$

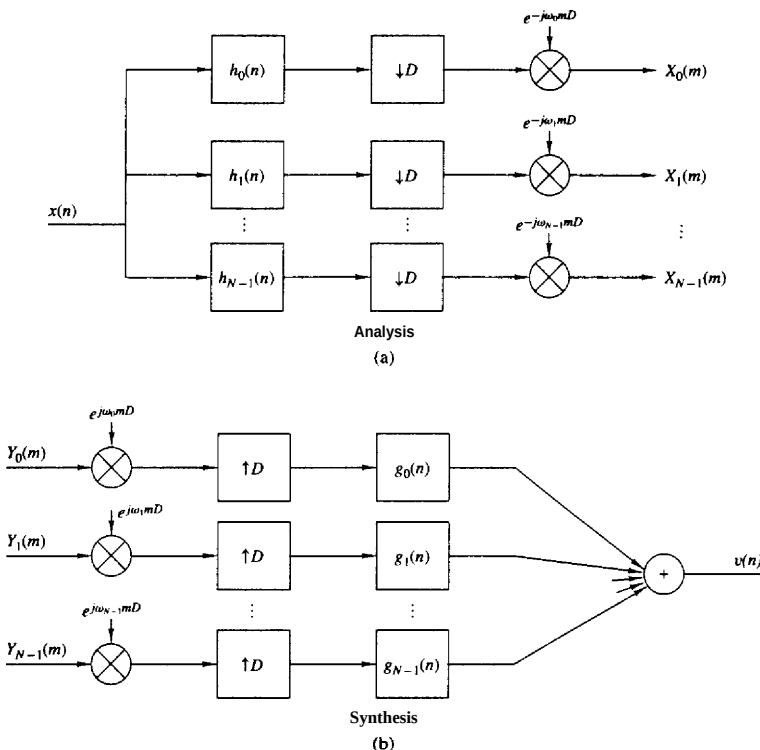


Figure 10.34 Alternative realization of a uniform DFT filter bank.

The output of each **bandpass** filter is decimated by a factor D and multiplied by $\exp(-j2\pi mk/N)$ to produce the DFT sequence $\{X_k(m)\}$. The modulation by the complex exponential allows us to shift the spectrum of the signal from $\omega_k = 2\pi k/N$ to $\omega_0 = 0$. Hence this realization is equivalent to the realization given in Fig. 10.33. The filter bank output can be written as

$$X_k(m) = \left[\sum_n x(n) h_0(mD - n) e^{j2\pi k(mD - n)/N} \right] e^{-j2\pi mkD/N} \quad (10.9.7)$$

The corresponding filter bank synthesizer can be realized as shown in Fig. 10.34b, where the input sequences are first multiplied by the exponential factors $[\exp(j2\pi kmD/N)]$, upsampled by the factor $I = D$, and the resulting se-

quences are filtered by the **bandpass** interpolation filters with impulse responses

$$g_k(n) = g_0(n)e^{j2\pi nk/N} \quad (10.9.8)$$

where $\{g_0(n)\}$ is the impulse response of the prototype filter. The outputs of these filters are then summed to yield

$$v(n) = \frac{1}{N} \sum_{k=0}^{N-1} \left\{ \sum_m [Y_k(m)e^{j2\pi kmI/N}] g_k(n - mI) \right\} \quad (10.9.9)$$

where $I = D$.

In the implementation of digital filters banks, computational efficiency can be achieved by use of polyphase filters for decimation and interpolation. Of particular interest is the case where the decimation factor D is selected to be equal to the number N of frequency bands. When $D = N$, we say that the filter bank is *critically sampled*.

For the analysis filter bank, let us define a set of $N = D$ polyphase filters with impulse responses

$$p_k(n) = h_0(nN - k) \quad k = 0, 1, \dots, N - 1 \quad (10.9.10)$$

and the corresponding set of decimated input sequences

$$x_k(n) = x(nN + k) \quad k = 0, 1, \dots, N - 1 \quad (10.9.11)$$

Note that this definition of $\{p_k(n)\}$ implies that the commutator for the **decimator** rotates clockwise.

The structure of the analysis filter bank based on the use of polyphase filters can be obtained by substituting (10.9.10) and (10.9.11) into (10.9.7) and rearranging the summation into the form

$$X_k(m) = \sum_{n=0}^{N-1} \left[\sum_l p_n(l) x_n(m-l) \right] e^{-j2\pi nk/N} \quad k = 0, 1, \dots, D-1 \quad (10.9.12)$$

where $N = D$. Note that the inner summation represents the convolution of $\{p_n(l)\}$ with $\{x_n(l)\}$. The outer summation represents the N -point DFT of the filter outputs. The filter structure corresponding to this computation is illustrated in Fig. 10.35. Each sweep of the commutator results in N outputs, denoted as $\{r_n(m), n = 0, 1, \dots, N-1\}$ from the N polyphase filters. The N -point DFT of this sequence yields the spectral samples $\{X_k(m)\}$. For large values of N , the FFT algorithm provides an efficient means for computing the DFT.

Now suppose that the spectral samples $\{X_k(m)\}$ are modified in some manner, prescribed by the application, to produce $\{Y_k(m)\}$. A filter bank synthesis filter based on a polyphase filter structure can be realized in a similar manner. First, we define the impulse response of the N ($D = I = N$) polyphase filters for the interpolation filter as

$$q_k(n) = g_0(nN + k) \quad k = 0, 1, \dots, N-1 \quad (10.9.13)$$

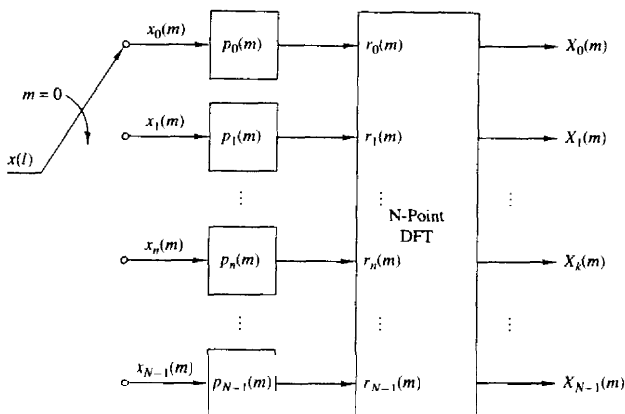


Figure 10.35 Digital filter bank structure for the computation of (10.9.12).

and the corresponding set of output signals as

$$v_k(n) = v(nN + k) \quad k = 0, 1, \dots, N-1 \quad (10.9.14)$$

Note that this definition of $\{q_l(n)\}$ implies that the commutator for the interpolator rotates counterclockwise.

By substituting (10.9.13) into (10.9.5), we can express the output $v_l(n)$ of the l th polyphase filter as

$$v_l(n) = \sum_m q_l(n-m) \left[\frac{1}{N} \sum_{k=0}^{N-1} Y_k(m) e^{j2\pi kl/N} \right] \quad l = 0, 1, \dots, N-1 \quad (10.9.15)$$

The term in brackets is the N-point inverse DFT of $\{Y_k(m)\}$, which we denote as $\{y_l(m)\}$. Hence

$$v_l(n) = \sum_m q_l(n-m) y_l(m) \quad l = 0, 1, \dots, N-1 \quad (10.9.16)$$

The synthesis structure corresponding to (10.9.16) is shown in Fig. 10.36. It is interesting to note that by defining the polyphase interpolation filter as in (10.9.13), the structure in Fig. 10.36 is the transpose of the polyphase analysis filter shown in Fig. 10.35.

In our treatment of digital filter banks we considered the important case of critically sampled DFT filter banks, where $D = N$. Other choices of D and N can be employed in practice, but the implementation of the filters becomes more complex. Of particular importance is the **oversampled** DFT filter bank, where $N = KD$, D denotes the decimation factor and K is an integer that specifies the

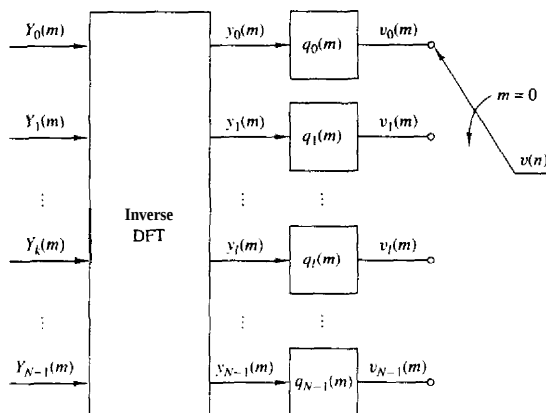


Figure 10.36 Digital filter bank structure for the computation of (10.9.16).

oversampling factor. In this case it can be shown that the polyphase filter bank structures for the analysis and synthesis filters can be implemented by use of N subfilters and N -point DFTs and inverse DFTs.

10.9.5 Subband Coding of Speech Signals

A variety of techniques have been developed to efficiently represent speech signals in digital form for either transmission or storage. Since most of the speech energy is contained in the lower frequencies, we would like to encode the lower-frequency band with more bits than the high-frequency band. **Subband** coding is a method, where the speech signal is subdivided into several frequency bands and each band is digitally encoded separately.

An example of a frequency subdivision is shown in Fig. 10.37a. Let us assume that the speech signal is sampled at a rate F_s samples per second. The first frequency subdivision **splits** the signal spectrum into two equal-width segments, a **lowpass** signal ($0 \leq F \leq F_s/4$) and a **highpass** signal ($F_s/4 \leq F \leq F_s/2$). The second frequency subdivision splits the **lowpass** signal from the first stage into two equal bands, a **lowpass** signal ($0 < F \leq F_s/8$) and a **highpass** signal ($F_s/8 \leq F \leq F_s/4$). Finally, the third frequency subdivision splits the **lowpass** signal from the second stage into two **equal** bandwidth signals. Thus the signal is subdivided into four frequency bands, covering three octaves, as shown in Fig. 10.37b.

Decimation by a factor of 2 is performed after frequency subdivision. By allocating a different number of bits per sample to the signal in the four subbands, we can achieve a reduction in the bit rate of the digitalized speech signal.

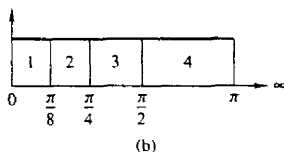
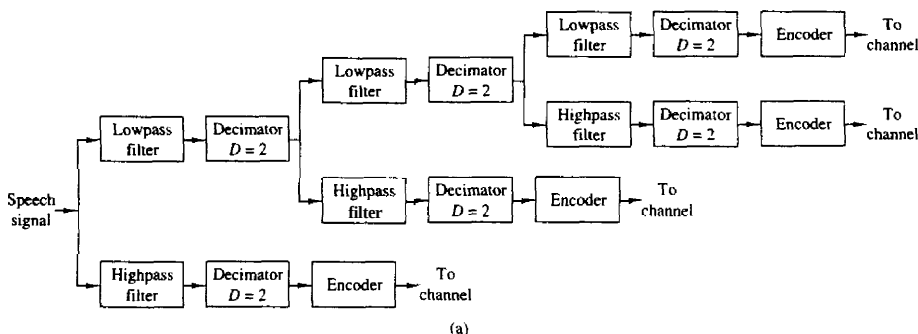


Figure 10.37 Block diagram of a subband speech coder.

Filter design is particularly important in achieving good performance in **subband** coding. Aliasing resulting from decimation of the **subband** signals must be negligible. It is clear that we cannot use brickwall filter characteristics as shown in Fig. 10.38a, since such filters are physically unrealizable. A particularly practical solution to the aliasing problem is to use **quadrature mirror** filters (QMF), which have the frequency response characteristics shown in Fig. 10.38b. These filters are described in the following section.

The synthesis method for the **subband** encoded speech signal is basically the reverse of the encoding process. The signals in adjacent **lowpass** and **highpass** frequency bands are interpolated, filtered, and combined as shown in Fig. 10.39. A pair of QMF is used in the signal synthesis for each octave of the signal.

Subband coding is also an effective method to achieve data compression in image signal processing. By combining **subband** coding with vector quantization for each **subband** signal, Safranek et al. (1988) have obtained coded images with approximately $\frac{1}{2}$ bit per pixel, compared with 8 bits per pixel for the uncoded image.

In general, **subband** coding of signals is an effective method for achieving bandwidth compression in a **digital** representation of the signal, when the signal energy is concentrated in a particular region of the frequency band. Multirate signal processing notions provide efficient implementations of the **subband** encoder.

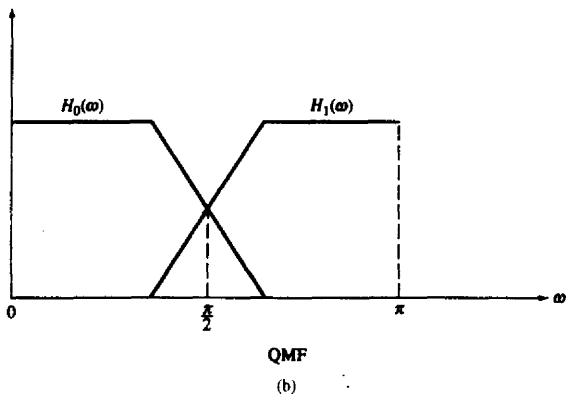
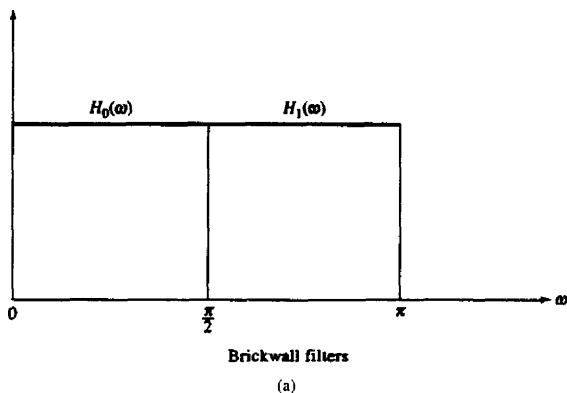


Figure 10.38 Filter characteristics for subband coding.

10.9.6 Quadrature Mirror Filters

The basic building block in applications of quadrature mirror filters (QMF) is the two-channel QMF bank shown in Fig. 10.40. This is a **multirate** digital filter structure that employs two **decimators** in the "signal analysis" section and two interpolators in the "signal synthesis" section. The **lowpass** and **highpass** filters in the analysis section have impulse responses $h_0(n)$ and $h_1(n)$, respectively. Similarly, the **lowpass** and **highpass** filters contained in the synthesis section have impulse responses $g_0(n)$ and $g_1(n)$, respectively.

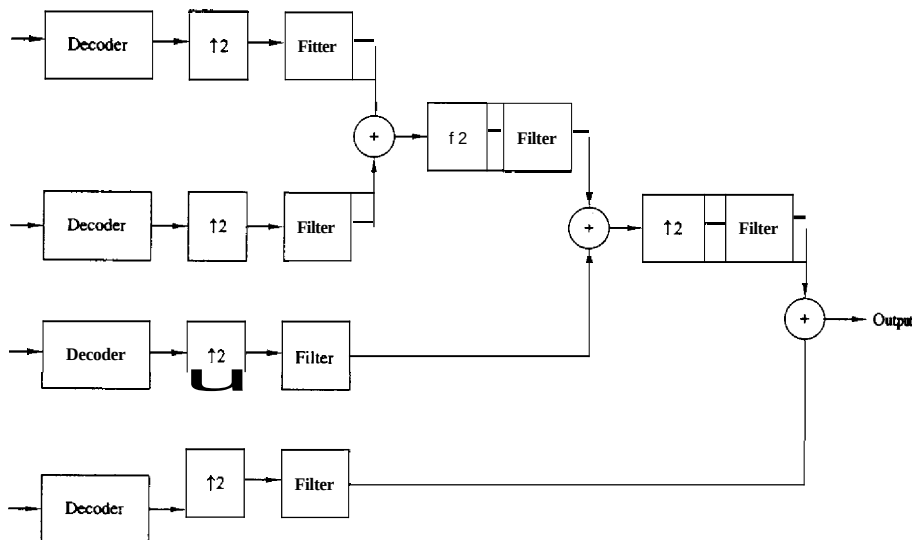


Figure 1039 Synthesis of subband-encoded signals.

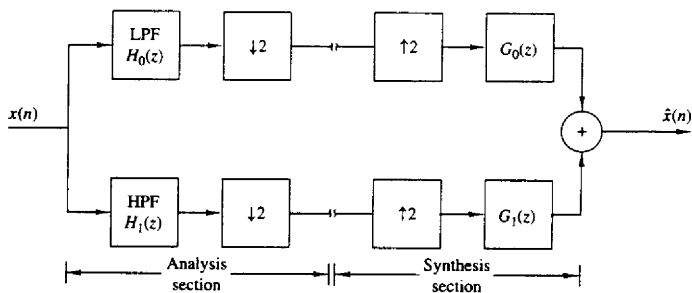


Figure 1040 Two-channel QMF bank.

The Fourier transforms of the signals at the outputs of the two decimators are

$$\begin{aligned}
 X_{a0}(\omega) &= \frac{1}{2} \left[X\left(\frac{\omega}{2}\right) H_0\left(\frac{\omega}{2}\right) + X\left(\frac{\omega - 2\pi}{2}\right) H_0\left(\frac{\omega - 2\pi}{2}\right) \right] \\
 X_{a1}(\omega) &= \frac{1}{2} \left[X\left(\frac{\omega}{2}\right) H_1\left(\frac{\omega}{2}\right) + X\left(\frac{\omega - 2\pi}{2}\right) H_1\left(\frac{\omega - 2\pi}{2}\right) \right]
 \end{aligned}
 \tag{10.9.17}$$

If $X_{s0}(\omega)$ and $X_{s1}(\omega)$ represent the two inputs to the synthesis section, the output

is simply

$$\hat{X}(\omega) = X_{s0}(2\omega)G_0(\omega) + X_{s1}(2\omega)G_1(\omega) \quad (10.9.18)$$

Now, suppose that we connect the analysis filter to the corresponding synthesis filter, so that $X_{s0}(\omega) = X_{s0}(\omega)$ and $X_{s1}(\omega) = X_{s1}(\omega)$. Then, by substituting from (10.9.17) into (10.9.18), we obtain

$$\begin{aligned} \hat{X}(\omega) = & \frac{1}{2}[H_0(\omega)G_0(\omega) + H_1(\omega)G_1(\omega)]X(\omega) \\ & + \frac{1}{2}[H_0(\omega - \pi)G_0(\omega) + H_1(\omega - \pi)G_1(\omega)]X(\omega - \pi) \end{aligned} \quad (10.9.19)$$

The first term in (10.9.19) is the desired signal output from the QMF bank. The second term represents the effect of aliasing, which we would like to eliminate. Hence we require that

$$H_0(\omega - \pi)G_0(\omega) + H_1(\omega - \pi)G_1(\omega) = 0 \quad (10.9.20)$$

This condition can be simply satisfied by selecting $G_0(\omega)$ and $G_1(\omega)$ as

$$G_0(\omega) = H_1(\omega - \pi) \quad G_1(\omega) = -H_0(\omega - \pi) \quad (10.9.21)$$

Thus the second term in (10.9.19) vanishes.

To elaborate, let us assume that $H_0(\omega)$ is a **lowpass** filter and $H_1(\omega)$ is a mirror-image **highpass** filter. Then we can express $H_0(\omega)$ and $H_1(\omega)$ as

$$\begin{aligned} H_0(\omega) &= H(\omega) \\ H_1(\omega) &= H(\omega - \pi) \end{aligned} \quad (10.9.22)$$

where $H(\omega)$ is the frequency response of a **lowpass** filter. In the time domain, the corresponding relations are

$$\begin{aligned} h_0(n) &= h(n) \\ h_1(n) &= (-1)^n h(n) \end{aligned} \quad (10.9.23)$$

As a consequence, $H_0(\omega)$ and $H_1(\omega)$ have mirror-image symmetry about the frequency $\omega = \pi/2$, as shown in Fig. 10.38b. To be consistent with the constraint in (10.9.21), we select the **lowpass** filter $G_0(\omega)$ as

$$G_0(\omega) = 2H(\omega) \quad (10.9.24)$$

and the **highpass** filter $G_1(\omega)$ as

$$G_1(\omega) = -2H(\omega - \pi) \quad (10.9.25)$$

In the time domain, these relations become

$$\begin{aligned} g_0(n) &= 2h(n) \\ g_1(n) &= -2(-1)^n h(n) \end{aligned} \quad (10.9.26)$$

The scale factor of 2 in $g_0(n)$ and $g_1(n)$ corresponds to the interpolation factor used to **normalize** the overall frequency response of the QMF. With this choice of

the filter characteristics, the component due to aliasing vanishes. Thus the aliasing resulting from decimation in the analysis section of the **QMF** bank is perfectly canceled by the image signal spectrum that arises due to interpolation. As a result, the two-channel QMF behaves as a linear, time-invariant system.

If we substitute for $H_0(\omega)$, $H_1(\omega)$, $G_0(\omega)$, and $G_1(\omega)$ into the first term of (10.9.19), we obtain

$$\hat{X}(\omega) = [H^2(\omega) - H^2(\omega - \pi)]X(\omega) \quad (10.9.27)$$

Ideally, the two-channel QMF bank should have unity gain,

$$|H^2(\omega) - H^2(\omega - \pi)| = 1 \quad \text{for all } \omega \quad (10.9.28)$$

where $H(\omega)$ is the frequency response of a **lowpass** filter. Furthermore, it is also desirable for the **QMF** to have linear phase.

Now, let us consider the use of a linear phase filter $H(\omega)$. Hence $H(\omega)$ may be expressed in the form

$$H(\omega) = H_r(\omega)e^{-j\omega(N-1)/2} \quad (10.9.29)$$

where N is the filter length. Then

$$\begin{aligned} H^2(\omega) &= H_r^2(\omega)e^{-j\omega(N-1)} \\ &= |H(\omega)|^2 e^{-j\omega(N-1)} \end{aligned} \quad (10.9.30)$$

and

$$\begin{aligned} H^2(\omega - \pi) &= H_r^2(\omega - \pi)e^{-j(\omega - \pi)(N-1)} \\ &= (-1)^{N-1} |H(\omega - \pi)|^2 e^{-j\omega(N-1)} \end{aligned} \quad (10.9.31)$$

Therefore, the **overall** transfer function of the two-channel QMF which employs linear-phase FIR filters is

$$\frac{\hat{X}(\omega)}{X(\omega)} = [|H(\omega)|^2 - (-1)^{N-1} |H(\omega - \pi)|^2] e^{-j\omega(N-1)} \quad (10.9.32)$$

Note that the overall filter has a delay of $N - 1$ samples and a magnitude characteristic

$$A(\omega) = |H(\omega)|^2 - (-1)^{N-1} |H(\omega - \pi)|^2 \quad (10.9.33)$$

We also note that when N is odd, $A(\pi/2) = 0$, because $|H(\pi/2)| = |H(3\pi/2)|$. **This is an** undesirable property for a QMF design. On the other hand, when N is even,

$$A(\omega) = |H(\omega)|^2 + |H(\omega - \pi)|^2 \quad (10.9.34)$$

which avoids the problem of a zero at $\omega = \pi/2$. For **N even**, the ideal two-channel QMF should satisfy the condition

$$A(\omega) = |H(\omega)|^2 + |H(\omega - \pi)|^2 = 1 \quad \text{for all } \omega \quad (10.9.35)$$

which follows from (10.9.33). Unfortunately, the only filter frequency response function that satisfies (10.9.35) is the trivial function $|H(\omega)|^2 = \cos^2 a\omega$. Consequently, any nontrivial linear-phase FIR filter $H(\omega)$ introduces some amplitude distortion.

The amount of amplitude distortion introduced by a nontrivial linear phase FIR filter in the QMF can be minimized by optimizing the FIR filter coefficients. A particularly effective method is to select the filter coefficients of $H(\omega)$ such that $A(\omega)$ is made as flat as possible while simultaneously minimizing (or constraining) the **stopband** energy of $H(\omega)$. This approach leads to the minimization of the integral squared error

$$J = w \int_{\omega_s}^{\pi} |H(\omega)|^2 d\omega + (1-w) \int_0^{\pi} [A(\omega) - 1]^2 d\omega \quad (10.9.36)$$

where w is a weighting factor in the range $0 < w < 1$. In **performing** the optimization, the filter impulse response is constrained to be symmetric (linear phase). This optimization is easily done numerically on a digital computer. This approach has been used by Johnston (1980), and Jain and Crochiere (1984) to design two-channel QMFs. Tables of optimum filter coefficients have been tabulated by Johnston (1980).

As an alternative to linear-phase FIR filters, we can design an IIR filter that satisfies the all-pass constraint given by (10.9.28). For this purpose, elliptic filters provide especially efficient designs. Since the QMF would introduce some phase distortion, the signal at the output of the QMF can be passed through an all-pass phase equalizer designed to minimize phase distortion.

In addition to these two methods for QMF design, one can also design the two-channel QMFs to eliminate completely both amplitude and phase distortion as well as canceling aliasing distortion. Smith and **Barnwell** (1984) have shown that such perfect reconstruction *QMF* can be designed by relaxing the linear-phase condition of the FIR **lowpass** filter $H(\omega)$. To achieve perfect reconstruction, we begin by designing a linear-phase **FIR halfband** filter of length $2N - 1$.

A half-band filter is defined as a zero-phase FIR filter whose impulse response $\{b(n)\}$ satisfies the condition

$$b(2n) = \begin{cases} \text{constant,} & n = 0 \\ 0, & n \neq 0 \end{cases} \quad (10.9.37)$$

Hence all the even-numbered samples are zero except at $n = 0$. The zero-phase requirement implies that $b(n) = b(-n)$. The frequency response of such a filter is

$$B(\omega) = \sum_{n=-K}^K b(n) e^{-j\omega n} \quad (10.9.38)$$

where K is odd. Furthermore, $B(\omega)$ satisfies the condition $B(\omega) + B(\pi - \omega)$ is equal to a constant for all frequencies. The typical frequency response characteristic of a half-band filter is shown in Fig. 10.41. We note that the filter response is symmetric with respect to $\pi/2$, the band edges frequencies ω_p and ω_s are symmetric about

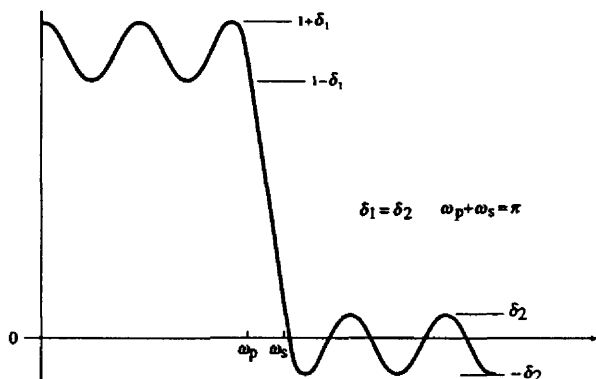


Figure 10.41 Frequency response characteristic of FIR half-band filter.

$\omega = \pi/2$, and the peak passband and stopband errors are equal. We also note that the filter can be made causal by introducing a delay of K samples.

Now, suppose that we design an FIR half-band filter of length $2N - 1$, where N is even, with frequency response as shown in Fig. 10.42(a). From $B(\omega)$ we construct another half-band filter with frequency response

$$B_+(\omega) = B(\omega) + \delta e^{-j\omega(N-1)} \quad (10.9.39)$$

as shown in Fig. 10.42(b). Note that $B_+(\omega)$ is nonnegative and hence it has the spectral factorization

$$B_+(z) = H(z)H(z^{-1})z^{-(N-1)} \quad (10.9.40)$$

or, equivalently,

$$B_+(\omega) = |H(\omega)|^2 e^{-j\omega(N-1)} \quad (10.9.41)$$

where $H(\omega)$ is the frequency response of an FIR filter of length N with real coefficients. Due to the symmetry of $B_+(\omega)$ with respect to $\omega = \pi/2$, we also have

$$B_+(z) + (-1)^{N-1} B_+(-z) = \alpha z^{-(N-1)} \quad (10.9.42)$$

or, equivalently,

$$B_+(\omega) + (-1)^{N-1} B_+(\omega - \pi) = \alpha e^{-j\omega(N-1)} \quad (10.9.43)$$

where α is a constant. Thus, by substituting (10.9.40) into (10.9.42), we obtain

$$H(z)H(z^{-1}) + H(-z)H(-z^{-1}) = \alpha \quad (10.9.44)$$

Since $H(z)$ satisfies (10.9.44) and since aliasing is eliminated when we have $G_0(z) = H_1(-z)$ and $G_1(z) = -H_0(-z)$, it follows that these conditions are satisfied by

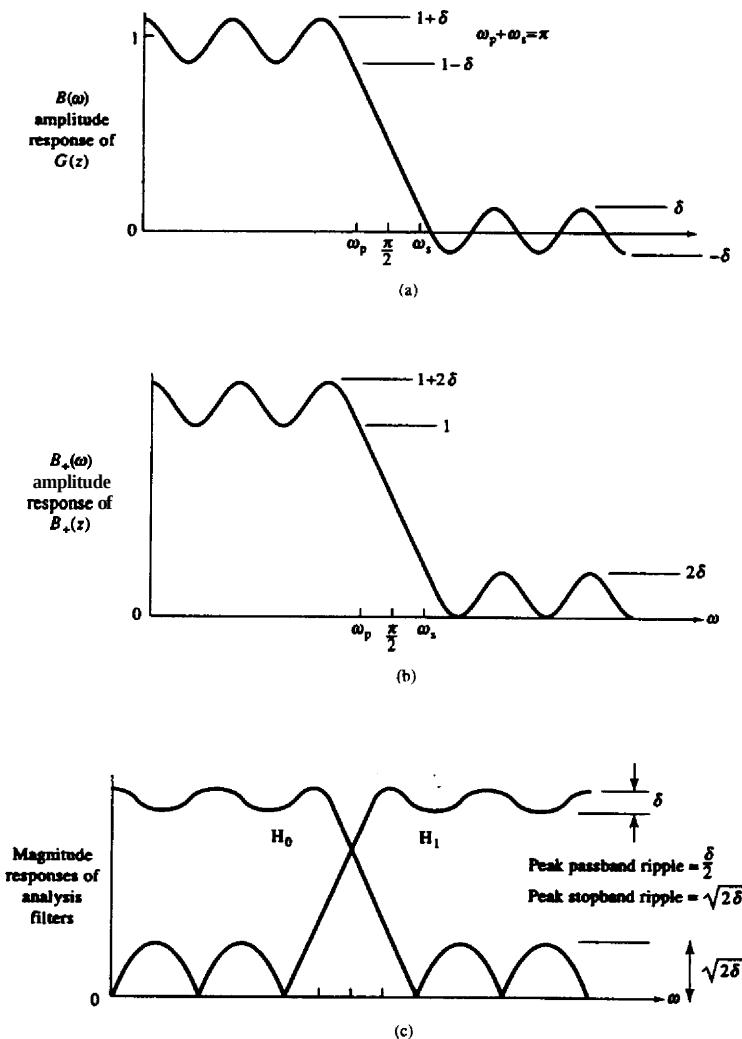


Figure 10.42 Frequency response characteristics of half-band filters $B(\omega)$ and $B_+(\omega)$. (From Vaidyanathan (1987))

choosing $H_1(z)$, $G_0(z)$, and $G_1(z)$ as

$$\begin{aligned} H_0(z) &= H(z) \\ H_1(z) &= -z^{-(N-1)} H_0(-z^{-1}) \\ G_0(z) &= z^{-(N-1)} H_0(z^{-1}) \\ G_1(z) &= z^{-(N-1)} H_1(z^{-1}) = -H_0(-z) \end{aligned} \quad (10.9.45)$$

Thus aliasing distortion is eliminated and since $\hat{X}(\omega)/X(\omega)$ is a constant, the QMF performs perfect reconstruction so that $x(n) = \alpha x(n - N + 1)$. However, we note that $H(z)$ is not a linear-phase filter.

The FIR filters $H_0(z)$, $H_1(z)$, $G_0(z)$, and $G_1(z)$ in the two-channel QMF bank are efficiently realized as polyphase filters. Since $I = D = 2$, two polyphase filters are implemented for each decimator and two for each interpolator. However, if we employ linear-phase **FIR** filters, the symmetry properties of the analysis filters and synthesis filters allow us to simplify the structure and reduce the number of polyphase filters in the analysis section to two filters and to another two filters in the synthesis section.

To demonstrate this construction, let us assume that the filters are **linear**-phase FIR filters of length N (N even), which have impulse responses given by (10.9.23). Then the outputs of the analysis filter pair, after decimation by a factor of 2, can be expressed as

$$\begin{aligned} X_{ak}(m) &= \sum_{n=-\infty}^{\infty} (-1)^{kn} h(n)x(2m - n) \quad k = 0, 1 \\ &= \sum_{i=0}^1 \sum_{l=-\infty}^{\infty} (-1)^{k(2l+1)} h(2l+1)x(2m - 2l - i) \\ &= \sum_{l=0}^{N/2-1} h(2l)x(2m - 2l) + (-1)^k \sum_{l=0}^{N/2-1} h(2l+1)x(2m - 2l - 1) \end{aligned} \quad (10.9.46)$$

Now let us define the impulse response of two polyphase filters of length $N/2$ as

$$p_i(m) = h(2m + i) \quad i = 0, 1 \quad (10.9.47)$$

Then (10.9.46) can be expressed as

$$\begin{aligned} X_{ak}(m) &= \sum_{l=0}^{N/2-1} p_0(m)x(2m - l) \\ &\quad + (-1)^k \sum_{l=0}^{N/2-1} p_1(m)x(2m - 2l - 1) \quad k = 0, 1 \end{aligned} \quad (10.9.48)$$

This expression corresponds to the polyphase filter structure for the analysis section shown in Fig. 10.43. Note that the commutator rotates counterclockwise

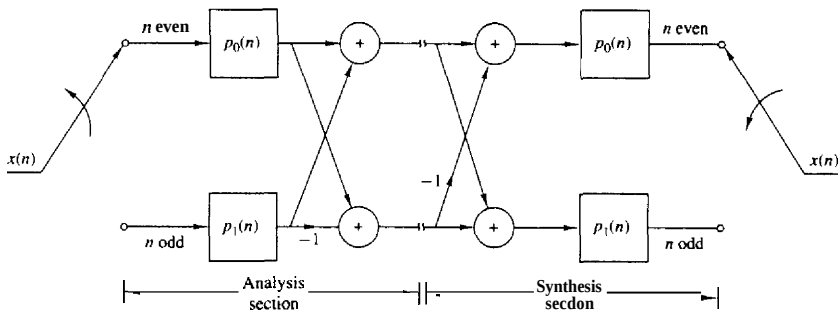


Figure 10.43 Polyphase filter structure for the QMF bank.

and that the filter with impulse response $\{p_0(m)\}$ processes the even-numbered samples of the input sequence and the filter with impulse response $\{p_1(m)\}$ processes the odd-numbered samples of the input signal.

In a similar manner, by using (10.9.26), we can obtain the structure for the polyphase synthesis section, which is also shown in Fig. 10.43. This derivation is left as an exercise for the reader (Problem 10.16). Note that the commutator also rotates counterclockwise.

Finally, we observe that the polyphase filter structure shown in Fig. 10.43 is approximately four times more efficient than the **direct-form** FIR filter realization.

10.9.7 Transmultiplexers

Another application of **multirate** signal processing is in the design and **implementation** of digital transmultiplexers which are devices for converting between **time-division-multiplexed** (TDM) signals and **frequency-division-multiplexed** (FDM) signals.

In a **transmultiplexer** for **TDM-to-FDM** conversion, the input signal $\{x(n)\}$ is a time-division multiplexed signal consisting of L signals, which are separated by a commutator switch. Each of these L signals are then modulated on different carrier frequencies to obtain an **FDM** signal for transmission. In a transmultiplexer for **FDM-to-TDM** conversion, the composite signal is separated by filtering into the L signal components which are then time-division multiplexed.

In telephony, single-sideband transmission is used with channels spaced at a nominal **4-kHz** bandwidth. Twelve channels are usually stacked in frequency to form a basic group channel, with a bandwidth of 48 kHz. Larger bandwidth **FDM** signals are formed by frequency translation of multiple groups into adjacent frequency bands. We shall confine our discussion to digital transmultiplexers for 12-channel FDM and **TDM** signals.

Let us first consider **FDM-to-TDM** conversion. The analog **FDM** signal is passed through an **A/D** converter as shown in Fig. 10.44a. The digital signal is then

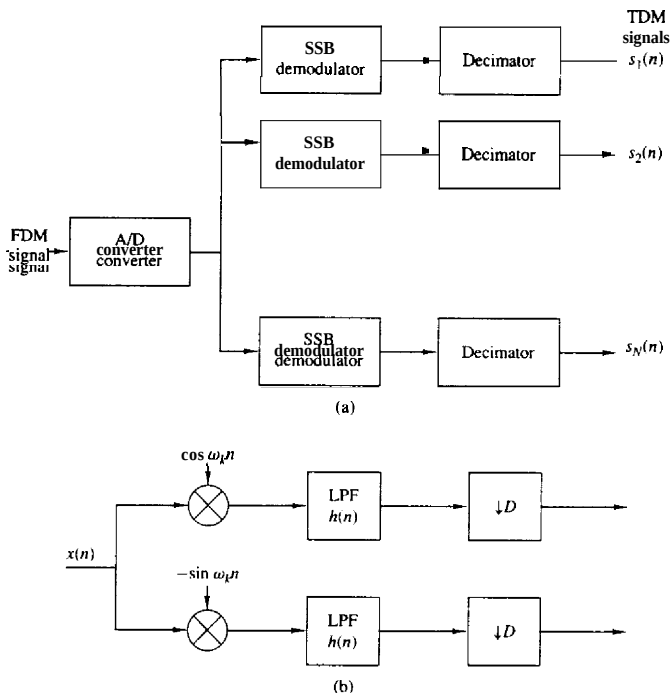


Figure 10.44 Block diagram of FDM-to-TDM transmultiplexer.

demodulated to baseband by means of single-sideband demodulators. The output of each demodulator is decimated and fed to commutator of the **TDM** system.

To be specific, let us assume that the 12-channel FDM signal is sampled at the Nyquist rate of 96 kHz and passed through a filter-bank demodulator. The basic building block in the FDM demodulator consists of a frequency converter, a **lowpass** filter, and a decimator, as illustrated in Fig. 10.44b. Frequency conversion can be efficiently implemented by the DFT filter bank described previously. The **lowpass** filter and decimator are efficiently implemented by use of the polyphase filter structure. Thus the basic structure for the FDM-to-TDM converter has the form of a DFT filter bank analyzer. Since the **signal** in each channel **occupies** a 4-kHz bandwidth, its Nyquist rate is 8 kHz, and hence the polyphase filter output can be decimated by a factor of 12. Consequently, the TDM commutator is operating at a rate of 12×8 kHz or 96 kHz.

In TDM-to-FDM conversion, the 12-channel TDM **signal** is demultiplexed into the 12 individual signals, where each signal has a rate of 8 kHz. The signal

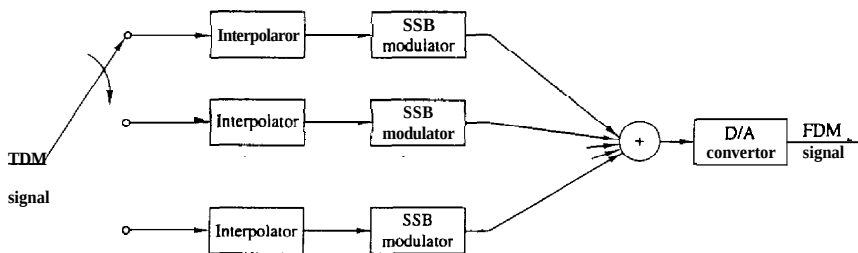


Figure 10.45 Block diagram of TDM-to-FDM transmultiplexer.

in each channel is interpolated by a factor of 12 and frequency converted by a single-sideband modulator, as shown in Fig. 10.45. The signal outputs from the 12 single-sideband modulators are summed and fed to the D/A converter. Thus we obtain the analog FDM signal for transmission. As in the case of FDM-to-TDM conversion, the interpolator and the modulator filter are combined and efficiently implemented by use of a polyphase filter. The frequency translation can be accomplished by the DFT. Consequently, the TDM-to-FDM converter encompasses the basic principles introduced previously in our discussion of DFT filter bank synthesis.

10.9.8 Oversampling A/D and D/A Conversion

Our treatment of oversampling A/D and D/A converters in Chapter 9 provides another example of **multirate** signal processing. Recall that an oversampling A/D converter is implemented by a cascade of an analog sigma-delta modulator (**SDM**) followed by a digital antialiasing decimation filter and a digital **highpass** filter as shown in Fig. 10.46. The analog SDM produces a 1-bit per sample output at a very high sampling rate. This 1-bit per sample output is passed through a digital **lowpass** filter, which provides a high-precision (multiple-bit) output that is decimated to a lower sampling rate. This output is then passed to a digital **highpass** filter that serves to attenuate the quantization noise at the lower frequencies.

The reverse operations take place in an oversampling D/A converter, as shown in Fig. 10.47. As illustrated in this figure, the digital signal is passed through a **highpass** filter whose output is fed to a digital interpolator (upsampler and anti-imaging filter). This high-sampling-rate signal is the input to the digital **SDM** that provides a high-sampling-rate 1-bit per sample output. The 1-bit per sample output

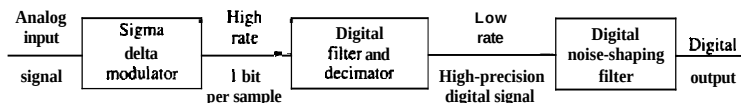


Figure 10.46 Diagram of oversampling AID converter

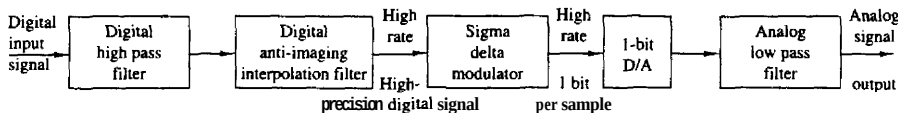


Figure 10.47 Diagram of oversampling D/A converter

is then converted to an analog signal by **lowpass** filtering and further smoothing with analog filters.

Figure 10.48 illustrates the block diagram of a commercial (Analog Devices **ADSP-28 msp02**) codec (encoder and decoder) for voice-band signals based on sigma-delta A/D and D/A converters and analog front-end circuits needed as an interface to the analog voice-band signals. The nominal sampling rate (after decimation) is 8 kHz and the sampling rate of the SDM is **1 MHz**. The codec has a 65-dB SNR and harmonic distortion performance.

10.10 SUMMARY AND REFERENCES

The need for sampling rate conversion arises **frequently** in digital signal processing applications. In this chapter we first treated sampling rate reduction (decimation) and **sampling** rate increase (interpolation) by integer factors and then demonstrated how the two processes can be combined to obtain sampling rate conversion by any rational factor. Later, in Section 10.8, we described a method to achieve sampling rate conversion by an arbitrary factor.

In general, the implementation of sampling rate conversion requires the use of a linear time-variant filter. We described methods for implementing such filters, including the class of polyphase filter structures, which are especially simple to implement. We also described the use of multistage implementations of multirate conversion as a means of simplifying the complexity of the filter required to meet the specifications.

In the special case where the signal to be resampled is a **bandpass** signal, we described two methods for performing the sampling rate conversion, one of which involves frequency conversion, while the second is a direct conversion method that does not employ modulation.

Finally, we described a number of applications that employ multirate signal processing, including the implementation of narrowband filters, phase **shifters**, filter banks, **subband** speech coders, and **transmultiplexers**. These are just a few of the many **applications** encountered in practice where multirate signal processing is used.

The first comprehensive treatment of multirate signal processing was given in the book by Crochiere and Rabiner (1983). In the technical literature, we cite the papers by Schafer and Rabiner (1973), and Crochiere and Rabiner (1975, 1976, 1981, 1983). **The** use of interpolation methods to achieve sampling rate conversion

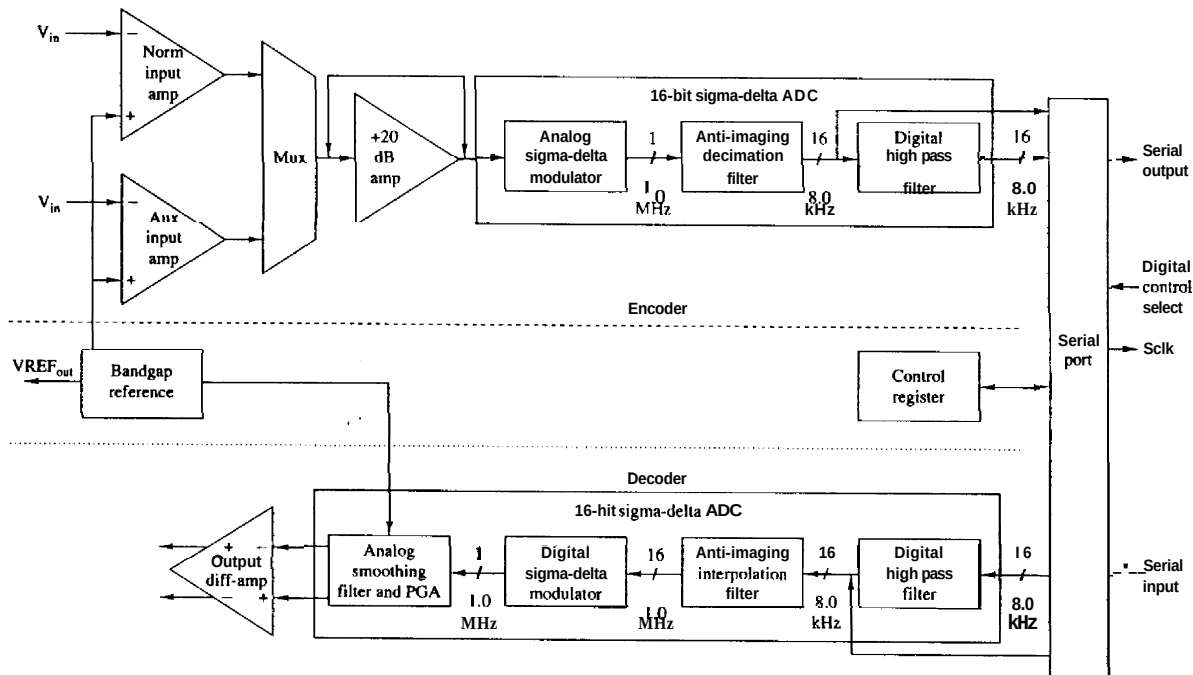


Figure 10.48 Diagram of Analog Devices ADSP-28 codec

by an arbitrary factor is treated in a paper by Ramstad (1984). A thorough tutorial treatment of multirate digital filters and filter banks, including quadrature mirror filters, is given by Vetterli (1987), and by Vaidyanathan (1990, 1993), where many references on various applications are cited. A comprehensive survey of digital transmultiplexing methods is found in the paper by Scheuermann and Gockler (1981). Subband coding of speech has been considered in many publications. The pioneering work on this topic was done by Crochiere (1977, 1981) and by Garland and Esteban (1980). Subband coding has also been applied to coding of images. We mention the papers by Vetterli (1984), Woods and O'Neil (1986), Smith and Eddins (1988), and Safranek et al. (1988) as just a few examples. In closing, we wish to emphasize that multirate signal processing continues to be a very active research area.

PROBLEMS

- 10.1** An analog signal $x_a(t)$ is bandlimited to the range $900 \leq F \leq 1100$ Hz. It is used as an input to the system shown in Fig. P10.1. In this system, $H(\omega)$ is an ideal lowpass filter with cutoff frequency $F_c = 125$ Hz.

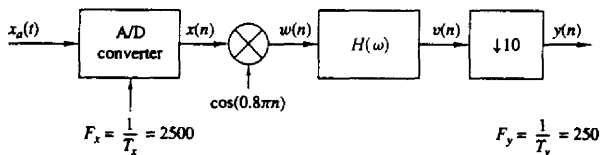


Figure P10.1

- (a) Determine and sketch the spectra for the signals $x(n)$, $w(n)$, $v(n)$, and $y(n)$.
 - (b) Show that it is possible to obtain $y(n)$ by sampling $x_a(t)$ with period $T = 4$ milliseconds.
- 10.2** Consider the signal $x(n) = a^n u(n)$, ($|a| < 1$).
- (a) Determine the spectrum $X(\omega)$.
 - (b) The signal $x(n)$ is applied to a decimator that reduces the rate by a factor of 2. Determine the output spectrum.
 - (c) Show that the spectrum in part (b) is simply the Fourier transform of $x(2n)$.
- 10.3** The sequence $x(n)$ is obtained by sampling an analog signal with period T . From this signal a new signal is derived having the sampling period $T/2$ by use of a linear interpolation method described by the equation

$$y(n) = \begin{cases} x(n/2), & n \text{ even} \\ \frac{1}{2} \left[x\left(\frac{n-1}{2}\right) + x\left(\frac{n+1}{2}\right) \right], & n \text{ odd} \end{cases}$$

- (a) Show that this linear interpolation scheme can be realized by basic digital signal processing elements.
- (b) Determine the spectrum of $y(n)$ when the spectrum of $x(n)$ is

$$X(\omega) = \begin{cases} 1, & 0 \leq |\omega| \leq 0.2\pi \\ 0, & \text{otherwise} \end{cases}$$

- (c) Determine the spectrum of $y(n)$ when the spectrum of $x(n)$ is

$$X(\omega) = \begin{cases} 1, & 0.7\pi \leq |\omega| \leq 0.9\pi \\ 0, & \text{otherwise} \end{cases}$$

10.4 Consider a signal $x(n]$ with Fourier transform

$$X(\omega) = 0 \text{ for } \begin{matrix} \omega_m < |\omega| \leq \pi \\ f_m < |f| \leq \frac{1}{2} \end{matrix}$$

- (a) Show that the signal $x(n)$ can be recovered from its samples $x(mD)$ if the sampling frequency $\omega_s = 2\pi/D \geq 2\omega_m$ ($f_s = 1/D \geq 2f_m$).
- (b) Show that $x(n)$ can be reconstructed using the formula

$$x(n) = \sum_{k=-\infty}^{\infty} x(kD)h_r(n-kD)$$

where

$$h_r(n) = \frac{\sin(2\pi f_c n)}{2\pi n} \quad \begin{matrix} f_m < f_c < f_s - f_m \\ \omega_m < \omega_c < \omega_s - \omega_m \end{matrix}$$

- (c) Show that the bandlimited interpolation in part (b) can be thought of as a two-step process: First, **increasing** the sampling rate by a factor of D by inserting $(D-1)$ zero samples between successive samples of the decimated signal $x_a(n) = x(nD)$ and second, filtering the resulting signal using an ideal **lowpass** filter with cutoff frequency ω_c .
- 105 In this problem we illustrate the concepts of sampling and decimation for **discrete-time** signals. To this end consider a signal $x(n)$ with Fourier **transform** $X(\omega)$ as in Fig. P10.5.

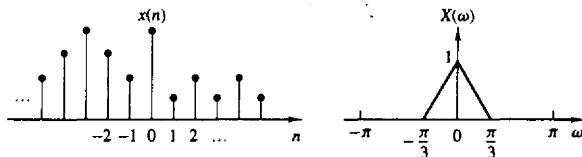


Figure P10.5

- (a) Sampling $x(n)$ with a sampling period $D = 2$ results to the signal

$$x_s(n) = \begin{cases} x(n), & n = 0, \pm 2, \pm 4, \dots \\ 0, & n = \pm 1, \pm 3, \pm 5, \dots \end{cases}$$

Compute and sketch the signal $x_s(n)$ and its Fourier **transform** $X_s(\omega)$. Can we reconstruct $x(n)$ from $x_s(n)$? How?

- (b) Decimating $x(n)$ by a factor of $D = 2$ produces the signal

$$x_d(n) = x(2n) \quad \text{all } n$$

Show that $X_d(\omega) = X_s(\omega/2)$. Plot the signal $x_d(n)$ and its transform $X_d(\omega)$. Do we lose any information when we decimate the sampled signal $x_s(n)$?

- 10.6*** Design a decimator that **downsamples** an input signal $x(n)$ by a factor $D = 5$. Use the Remez algorithm to determine the coefficients of the FIR filter that has 0.1-dB ripple in the **passband** ($0 \leq \omega \leq \pi/5$) and is down by at least 30 dB in the stopband. Also determine the corresponding polyphase filter structure for implementing the decimator.
- 10.7*** Design an interpolator that increases the input sampling rate by a factor of $I = 2$. Use the Remez algorithm to determine the coefficients of the FIR filter that has a 0.1-dB ripple in the **passband** ($0 \leq \omega \leq \pi/2$) and is down by at least 30 dB in the stopband. Also, determine the corresponding polyphase filter structure for implementing the interpolator.
- 10.8*** Design a sample-rate converter that reduces the sampling rate by a factor $\frac{2}{3}$. Use the Remez algorithm to determine the coefficients of the FIR filter that has a 0.1-dB ripple in the **passband** and is down by at least 30 dB in the stopband. Specify the sets of time-variant coefficients $g(n, m)$ and the corresponding coefficients in the polyphase filter realization of the sample-rate converter.
- 10.9** Consider the two different ways of cascading a decimator with an interpolator shown in Fig. P10.9.

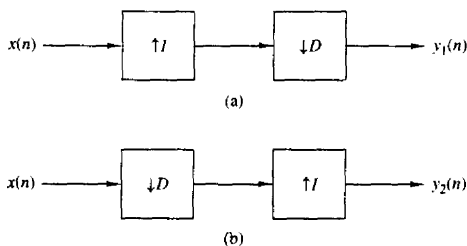


Figure P10.9

- (a) If $D = I$, show that the outputs of the two configurations are different. Hence, in general, the two systems are not identical.
- (b) Show that the two systems are identical if and only if D and I are relatively prime.
- 10.10** Prove the equivalence of the two decimator and interpolator configurations shown in Fig. P10.10. These equivalent relations are called the "noble identities" (see Vaidyanathan, 1990).
- 10.11** Consider an arbitrary digital filter with transfer function

$$H(z) = \sum_{n=-\infty}^{\infty} h(n)z^{-n}$$

- (a) Perform a **two-component** polyphase decomposition of $H(z)$ by grouping the even-numbered samples $h_0(n) = h(2n)$ and the odd-numbered samples $h_1(n) =$

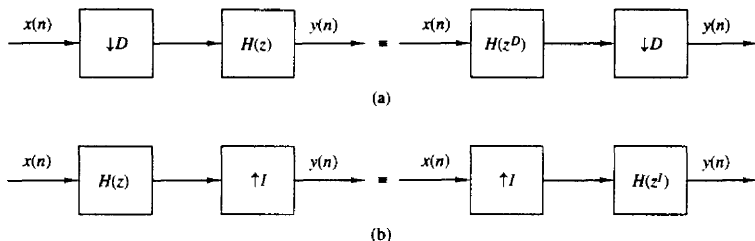


Figure P10.10

$h(2n+1)$. Thus show that $H(z)$ can be expressed as

$$H(z) = H_0(z^2) + z^{-1}H_1(z^2)$$

and determine $H_0(z)$ and $H_1(z)$.

- (b) Generalize the result in part (a) by showing that $H(z)$ can be decomposed into an D -component polyphase filter structure with transfer function

$$H(z) = \sum_{k=0}^{D-1} z^{-k} H_k(z^D)$$

Determine $H_k(z)$.

- (c) For the IIR filter with transfer function

$$H(z) = \frac{1}{1 - az^{-1}}$$

determine $H_0(z)$ and $H_1(z)$ for the two-component decomposition.

- 10.12 Design a two-stage decimator for the following specifications

	$D = 100$
Passband:	$0 \leq F \leq 50$
Transition band:	$50 \leq F \leq 55$
Input sampling rate:	10,000 Hz
Ripple:	$\delta_1 = 10^{-1}, \delta_2 = 10^{-3}$

- 10.13 Design a linear phase **FIR** filter that satisfies the following specifications based on a single-stage and a two-stage **multirate** structure.

Sampling rate:	10,000 Hz
Passband:	$0 \leq F \leq 60$
Transition band:	$60 \leq F \leq 65$
Ripple:	$\delta_1 = 10^{-1}, \delta_2 = 10^{-3}$

- 10.14 Prove that the half-band filter that satisfies (10.9.43) is always odd and the even coefficients are zero.

10.15 Design one-stage and two-stage interpolators to meet the following specification:

	$I = 20$
Input sampling rate:	10,000 Hz
Passband:	$0 \leq F \leq 90$
Transition band:	$90 \leq F \leq 100$
Ripple:	$\delta_1 = 10^{-2}, \delta_2 = 10^{-3}$

10.16 By using (10.9.26) derive the equations corresponding to the structure for the polyphase synthesis section shown in Fig. 10.43.

10.17 Show that the transpose of an L -stage interpolator for increasing the sampling rate by an integer factor I is equivalent to an L -stage decimator that decreases the sampling rate by a factor $D = I$.

10.18 Sketch the polyphase filter structure for achieving a time advance of $(k/I)T_s$ in a sequence $x(n)$.

10.19 Prove the following expressions for an interpolator of order I .

(a) The impulse response $h(n)$ can be expressed as

$$h(n) = \sum_{k=0}^{I-1} p_k(n-k)$$

where

$$p_k(n) = \begin{cases} p_k(n/I), & n = 0, \pm I, \pm 2I, \dots \\ 0, & \text{otherwise} \end{cases}$$

(b) $H(z)$ may be expressed as

$$H(z) = \sum_{k=0}^{I-1} z^{-k} p_k(z)$$

$$(c) \quad P_k(z) = \frac{1}{I} \sum_{n=-\infty}^{\infty} \sum_{l=0}^{I-1} h(n) e^{j2\pi l(n-k)/I} z^{-(n-k)/I}$$

$$P_k(\omega) = \frac{1}{I} \sum_{l=0}^{I-1} H\left(\omega - \frac{2\pi l}{I}\right) e^{j(\omega - 2\pi l)k/I}$$

10.20* Zoom-frequency analysis Consider the system in Fig. P10.20a.

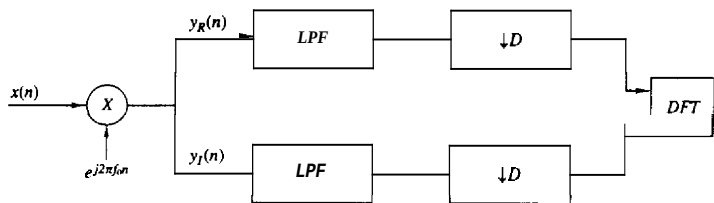
(a) Sketch the spectrum of the signal $y(n) = y_R(n) + jy_I(n)$ if the input signal $x(n)$ has the spectrum shown in Fig. P10.20b.

(b) Suppose that we are interested in the analysis of the frequencies in the band $f_0 \leq f \leq f_0 + \Delta f$, where $f_0 = \pi/6$ and $\Delta f = \pi/3$. Determine the cutoff of a lowpass filter and the decimation factor D required to retain the information contained in this band of frequencies.

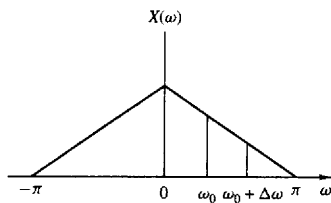
(c) Assume that

$$x(n) = \sum_{k=0}^{p-1} \left(1 - \frac{k}{2p}\right) \cos 2\pi f_k n$$

where $p = 40$ and $f_k = k/p$, $k = 0, 1, \dots, p-1$. Compute and plot the 1024-point DFT of $x(n)$.



(a)



(b)

Figure P10.19

- (d) Repeat part (b) for the signal $x(n)$ given in part (c) by using an appropriately designed lowpass linear phase FIR filter to determine the decimated signal $s(n) = s_R(n) + js_I(n)$.
- (e) Compute the 1024-point DFT of $s(n)$ and investigate to see if you have obtained the expected results.



11

Linear Prediction and Optimum Linear Filters

The design of filters to perform signal estimation is a problem that frequently arises in the design of communication systems, control systems, in geophysics, and in many other applications and disciplines. In this chapter we treat the problem of optimum filter design from a statistical viewpoint. The filters are constrained to be linear and the optimization criterion is based on the minimization of the mean-square error. As a consequence, only the second-order statistics (**autocorrelation** and **crosscorrelation** functions) of a stationary process are required in the determination of the optimum filters.

Included in this treatment is the design of optimum filters for linear prediction. Linear prediction is a particularly important topic in digital signal processing, with applications in a variety of areas, such as speech signal processing, image processing, and noise suppression in communication systems. As we shall observe, **determination** of the optimum linear filter for prediction requires the solution of a set of linear equations that have some special symmetry. To solve these linear equations, we describe two algorithms, the **Levinson-Durbin** algorithm and the Schur algorithm, which provide the solution to the equations through **computationally** efficient procedures that exploit the symmetry properties.

The last section of the chapter treats an important class of optimum filters called Wiener filters. Wiener filters are widely used in many applications involving the estimation of signals corrupted with additive noise.

11.1 INNOVATIONS REPRESENTATION OF A STATIONARY RANDOM PROCESS

In this section we demonstrate that a wide-sense stationary random process can be represented as the output of a causal and causally invertible linear system excited by a white noise process. The condition that the system is causally invertible also allows us to represent the wide-sense stationary random process by the output of the inverse system, which is a white noise process.

Let us consider a wide-sense stationary process $\{x(n)\}$ with autocorrelation sequence $\{\gamma_{xx}(m)\}$ and power spectral density $\Gamma_{xx}(f)$, $|f| \leq \frac{1}{2}$. We assume that $\Gamma_{xx}(f)$ is real and continuous for all $|f| \leq \frac{1}{2}$. The z-transform of the autocorrelation sequence $\{\gamma_{xx}(m)\}$ is

$$\Gamma_{xx}(z) = \sum_{m=-\infty}^{\infty} \gamma_{xx}(m) z^{-m} \quad (11.1.1)$$

from which we obtain the power spectral density by evaluating $\Gamma_{xx}(z)$ on the unit circle [i.e. by substituting $z = \exp(j2\pi f)$].

Now, let us assume that $\log \Gamma_{xx}(z)$ is analytic (possesses derivatives of all orders) in an annular region in the z-plane that includes the unit circle (i.e., $r_1 < |z| < r_2$ where $r_1 < 1$ and $r_2 > 1$). Then, $\log \Gamma_{xx}(z)$ can be expanded in a Laurent series of the form

$$\log \Gamma_{xx}(z) = \sum_{m=-\infty}^{\infty} v(m) z^{-m} \quad (11.1.2)$$

where the $\{v(m)\}$ are the coefficients in the series expansion. We can view $\{v(m)\}$ as the sequence with z-transform $V(z) = \log \Gamma_{xx}(z)$. Equivalently, we can evaluate $\log \Gamma_{xx}(z)$ on the unit circle,

$$\log \Gamma_{xx}(f) = \sum_{m=-\infty}^{\infty} v(m) e^{-j2\pi f m} \quad (11.1.3)$$

so that the $\{v(m)\}$ are the Fourier coefficients in the Fourier series expansion of the periodic function $\log \Gamma_{xx}(f)$. Hence

$$v(m) = \int_{-\frac{1}{2}}^{\frac{1}{2}} [\log \Gamma_{xx}(f)] e^{j2\pi f m} df \quad m = 0, \pm 1, \dots \quad (11.1.4)$$

We observe that $v(m) = v(-m)$, since $\Gamma_{xx}(f)$ is a real and even function of f .

From (11.1.2) it follows that

$$\begin{aligned} \Gamma_{xx}(z) &= \exp \left[\sum_{m=-\infty}^{\infty} v(m) z^{-m} \right] \\ &= \sigma_w^2 H(z) H(z^{-1}) \end{aligned} \quad (11.1.5)$$

where, by definition, $\sigma_w^2 = \exp[v(0)]$ and

$$H(z) = \exp \left[\sum_{m=1}^{\infty} v(m) z^{-m} \right], \quad |z| > r_1 \quad (11.1.6)$$

If (11.1.5) is evaluated on the unit circle, we have the equivalent representation of the power spectral density as

$$\Gamma_{xx}(f) = \sigma_w^2 |H(f)|^2 \quad (11.1.7)$$

We note that

$$\begin{aligned} \log \Gamma_{xx}(f) &= \log \sigma_w^2 + \log H(f) + \log H^*(f) \\ &= \sum_{m=-\infty}^{\infty} v(m) e^{-j2\pi f m} \end{aligned}$$

From the definition of $H(z)$ given by (11.1.6), it is clear that the causal part of the Fourier series in (11.1.3) is associated with $H(z)$ and the **anticausal** part is associated with $H(z^{-1})$. The Fourier series coefficients $\{v(m)\}$ are the **cepstral coefficients** and the sequence $\{v(m)\}$ is called the cepstrum of the sequence $\{\gamma_{xx}(m)\}$, as defined in Section 4.2.7.

The filter with system function $H(z)$ given by (11.1.6) is analytic in the region $|z| > r_1 < 1$. Hence, in this region, it has a Taylor series expansion as a causal system of the form

$$H(z) = \sum_{m=0}^{\infty} h(m)z^{-m} \quad (11.1.8)$$

The output of this filter in response to a white noise input sequence $w(n)$ with power spectral density σ_w^2 is a stationary random process $\{x(n)\}$ with power spectral density $\Gamma_{xx}(f) = \sigma_w^2 |H(f)|^2$. Conversely, the stationary random process $\{x(n)\}$ with power spectral density $\Gamma_{xx}(f)$ can be **transformed** into a white noise process by passing $\{x(n)\}$ through a linear filter with system function $1/H(z)$. We call this filter a noise **whitening filter**. Its output, denoted as $\{w(n)\}$ is called the **innovations process** associated with the stationary random process $\{x(n)\}$. These two relationships are illustrated in Fig. 11.1.

The representation of the stationary stochastic process $\{x(n)\}$ as the output of an IIR filter with system function $H(z)$ given by (11.1.8) and excited by a white noise sequence $\{w(n)\}$ is called the **Wold representation**.

11.1.1 Rational Power Spectra

Let us now restrict our attention to the case where the power spectral density of the stationary random process $\{x(n)\}$ is a rational function, expressed as

$$\Gamma_{xx}(z) = \sigma_w^2 \frac{B(z)B(z^{-1})}{A(z)A(z^{-1})} \quad r_1 < |z| < r_2 \quad (11.1.9)$$

where the polynomials $B(z)$ and $A(z)$ have roots that fall inside the unit circle in the z -plane. Then the linear filter $H(z)$ for generating the random process $\{x(n)\}$

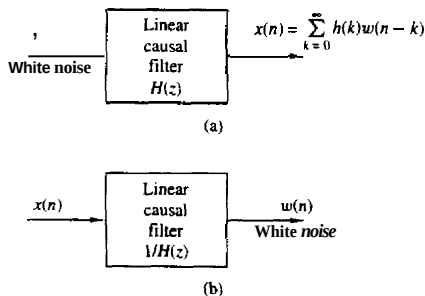


Figure 11.1 Filters for generating (a) the random process $x(n)$ from white noise and (b) the inverse filter.

from the white noise sequence $\{w(n)\}$ is also rational and is expressed as

$$H(z) = \frac{B(z)}{A(z)} = \frac{\sum_{k=0}^q b_k z^{-k}}{1 + \sum_{k=1}^p a_k z^{-k}} \quad |z| > r_1 \quad (11.1.10)$$

where $\{b_k\}$ and $\{a_k\}$ are the filter coefficients that **determine** the location of the zeros and poles of $H(z)$, respectively. Thus $H(z)$ is causal, stable, and minimum phase. Its reciprocal $1/H(z)$ is also a causal, stable, and minimum-phase linear system. Therefore, the random process $\{x(n)\}$ uniquely represents the statistical properties of the innovations process $\{w(n)\}$, and vice versa.

For the linear system with the rational system function $H(z)$ given by (11.1.10), the output $x(n)$ is related to the input $w(n)$ by the difference equation

$$x(n) + \sum_{k=1}^p a_k x(n-k) = \sum_{k=0}^q b_k w(n-k) \quad (11.1.11)$$

We will distinguish among three specific cases.

Autoregressive (AR) process. $b_0 = 1, b_k = 0, k > 0$. In this case, the linear filter $H(z) = 1/A(z)$ is an all-pole filter and the difference equation for the input-output relationship is

$$x(n) + \sum_{k=1}^p a_k x(n-k) = w(n) \quad (11.1.12)$$

In turn, the noise-whitening filter for generating the innovations process is an all-zero filter.

Moving average (MA) process. $a_k = 0, k \geq 1$. In this case, the linear filter $H(z) = B(z)$ is an all-zero filter and the difference equation for the input-output relationship is

$$x(n) = \sum_{k=0}^q b_k w(n-k) \quad (11.1.13)$$

The noise-whitening filter for the MA process is an all-pole filter.

Autoregressive, moving average (ARMA) process. In this case, the linear filter $H(z) = B(z)/A(z)$ has both finite poles and zeros in the z -plane and the corresponding difference equation is given by (11.1.11). The inverse system for generating the innovation process from $x(n)$ is also a pole-zero system of the form $1/H(z) = A(z)/B(z)$.

11.1.2 Relationships Between the Filter Parameters and the Autocorrelation Sequence

When the power spectral density of the stationary random process is a rational function, there is a basic relationship between the autocorrelation sequence

$\{\gamma_{xx}(m)\}$ and the parameters $\{a_k\}$ and $\{b_k\}$ of the linear filter $H(z)$ that generates the process by filtering the white noise sequence $w(n)$. This relationship can be obtained by multiplying the difference equation in (11.1.11) by $x^*(n-m)$ and taking the expected value of both sides of the resulting equation. Thus we have

$$\begin{aligned} E[x(n)x^*(n-m)] &= -\sum_{k=1}^p a_k E[x(n-k)x^*(n-m)] \\ &\quad + \sum_{k=0}^q b_k E[w(n-k)x^*(n-m)] \end{aligned} \quad (11.1.14)$$

Hence

$$\gamma_{xx}(m) = -\sum_{k=1}^p a_k \gamma_{xx}(m-k) + \sum_{k=0}^q b_k \gamma_{wx}(m-k) \quad (11.1.15)$$

where $\gamma_{wx}(m)$ is the cross-correlation sequence between $w(n)$ and $x(n)$.

The crosscorrelation $\gamma_{wx}(m)$ is related to the filter impulse response. That is,

$$\begin{aligned} \gamma_{wx}(m) &= E[x^*(n)w(n+m)] \\ &= E\left[\sum_{k=0}^{\infty} h(k)w^*(n-k)w(n+m)\right] \\ &= \sigma_w^2 h(-m) \end{aligned} \quad (11.1.16)$$

where, in the last step, we have used the fact that the sequence $w(n)$ is white. Hence

$$\gamma_{wx}(m) = \begin{cases} 0, & m > 0 \\ \sigma_w^2 h(-m), & m \leq 0 \end{cases} \quad (11.1.17)$$

By combining (11.1.17) with (11.1.15), we obtain the desired relationship

$$\gamma_{xx}(m) = \begin{cases} -\sum_{k=1}^p a_k \gamma_{xx}(m-k), & m > q \\ -\sum_{k=1}^p a_k \gamma_{xx}(m-k) + \sigma_w^2 \sum_{k=0}^{q-m} h(k)b_{k+m}, & 0 \leq m \leq q \\ \gamma_{xx}^*(-m), & m < 0 \end{cases} \quad (11.1.18)$$

This represents a nonlinear relationship between $\gamma_{xx}(m)$ and the parameters $\{a_k\}$, $\{b_k\}$.

The relationship in (11.1.18) applies, in general, to the ARMA process. For an AR process, (11.1.18) simplifies to

$$\gamma_{xx}(m) = \begin{cases} -\sum_{k=1}^p a_k \gamma_{xx}(m-k), & m > 0 \\ -\sum_{k=1}^p a_k \gamma_{xx}(m-k) + \sigma_w^2, & m = 0 \\ \gamma_{xx}^*(-m), & m < 0 \end{cases} \quad (11.1.19)$$

Thus we have a linear relationship between $\gamma_{xx}(m)$ and the $\{a_k\}$ parameters. These equations, called the Yule–Walker equations, can be expressed in the matrix form

$$\begin{bmatrix} \gamma_{xx}(0) & \gamma_{xx}(-1) & \gamma_{xx}(-2) & \cdots & \gamma_{xx}(-p) \\ \gamma_{xx}(1) & \gamma_{xx}(0) & \gamma_{xx}(-1) & \cdots & \gamma_{xx}(-p+1) \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ \gamma_{xx}(p) & \gamma_{xx}(p-1) & \gamma_{xx}(p-2) & \cdots & \gamma_{xx}(0) \end{bmatrix} \begin{bmatrix} 1 \\ a_1 \\ \vdots \\ a_p \end{bmatrix} = \begin{bmatrix} \sigma_w^2 \\ 0 \\ \vdots \\ 0 \end{bmatrix} \quad (11.1.20)$$

This correlation matrix is **Toeplitz**, and hence it can be efficiently inverted by use of the algorithms described in Section 11.3.

Finally, by setting $a_k = 0$, $1 \leq k \leq p$, and $h(k) = b_k$, $0 \leq k \leq q$, in (11.1.18), we obtain the relationship for the autocorrelation sequence in the case of a MA process, namely,

$$\gamma_{xx}(m) = \begin{cases} \sigma_w^2 \sum_{k=0}^q b_k b_{k+m}, & 0 \leq m \leq q \\ 0, & m > q \\ \gamma_{xx}^*(-m), & m < 0 \end{cases} \quad (11.1.21)$$

11.2 FORWARD AND BACKWARD LINEAR PREDICTION

Linear prediction is an important topic in digital signal processing with many practical applications. In this section we consider the problem of linearly predicting the value of a stationary random process either forward in time or backward in time. This formulation leads to lattice filter structures and to some interesting connections to parametric signal models.

11.2.1 Forward Linear Prediction

Let us begin with the problem of predicting a future value of a stationary random process from observation of past values of the process. In particular, we consider the **one-step forward linear predictor**, which **forms** the prediction of the value $x(n)$ by a weighted linear combination of the past values $x(n-1), x(n-2), \dots, x(n-p)$. Hence the linearly predicted value of $x(n)$ is

$$\hat{x}(n) = - \sum_{k=1}^p a_p(k) x(n-k) \quad (11.2.1)$$

where the $\{-a_p(k)\}$ represent the weights in the linear combination. These weights are called the **prediction coefficients** of the one-step forward linear predictor of **order** p . The negative sign in the definition of $\hat{x}(n)$ is for mathematical convenience and conforms with current practice in the technical literature.

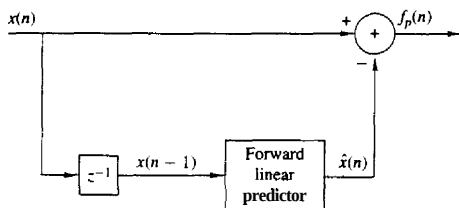


Figure 11.3 Forward linear prediction

The difference between the value $x(n)$ and the predicted value $\hat{x}(n)$ is called the **forward prediction error**, denoted as $f_p(n)$:

$$\begin{aligned} f_p(n) &= x(n) - \hat{x}(n) \\ &= x(n) + \sum_{k=1}^p a_p(k)x(n-k) \end{aligned} \quad (11.2.2)$$

We view linear prediction as equivalent to linear filtering where the predictor is embedded in the linear filter, as shown in Fig. 11.2. This is called a **prediction-error filter** with input sequence $\{x(n)\}$ and output sequence $\{f_p(n)\}$. An equivalent realization for the prediction-error filter is shown in Fig. 11.3. This realization is a direct-form FIR filter with system function

$$A_p(z) = \sum_{k=0}^p a_p(k)z^{-k} \quad (11.2.3)$$

where, by definition, $a_p(0) = 1$.

As shown in Section 7.2.4, the direct-form FIR filter is equivalent to an **all-zero lattice filter**. The lattice filter is generally described by the following set of

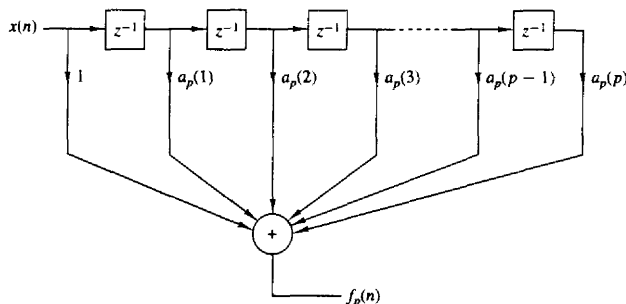


Figure 11.3 Prediction-error filter

order-recursive equation:

$$f_0(n) = g_0(n) = x(n)$$

$$f_m(n) = f_{m-1}(n) + K_m g_{m-1}(n-1) \quad m = 1, 2, \dots, p \quad (11.2.4)$$

$$g_m(n) = K_m^* f_{m-1}(n) + g_{m-1}(n-1) \quad m = 1, 2, \dots, p$$

where $\{K_m\}$ are the reflection coefficients and $g_m(n)$ is the backward prediction error defined in the following section. Note that for complex-valued data, the conjugate of K_m is used in the equation for $g_m(n)$. Figure 11.4 illustrates a p-stage lattice filter in block diagram form along with a typical stage showing the computations given by (11.2.4).

As a consequence of the equivalence between the direct-form prediction-error FIR filter and the FIR lattice filter, the output of the p-stage lattice filter is expressed as

$$f_p(n) = \sum_{k=0}^p a_p(k)x(n-k) \quad a_p(0) = 1 \quad (11.2.5)$$

Since (11.2.5) is a convolution sum, the z-transform relationship is

$$F_p(z) = A_p(z)X(z) \quad (11.2.6)$$

or, equivalently,

$$A_p(z) = \frac{F_p(z)}{X(z)} = \frac{F_p(z)}{F_0(z)} \quad (11.2.7)$$

The mean-square value of the forward linear prediction error $f_p(n)$ is

$$\begin{aligned} \mathcal{E}_p^f &= E[|f_p(n)|^2] \\ &= \gamma_{xx}(0) + 2 \operatorname{Re} \left[\sum_{k=1}^p a_p^*(k) \gamma_{xx}(k) \right] + \sum_{k=1}^p \sum_{l=1}^p a_p^*(l) a_p(k) \gamma_{xx}(l-k) \end{aligned} \quad (11.2.8)$$

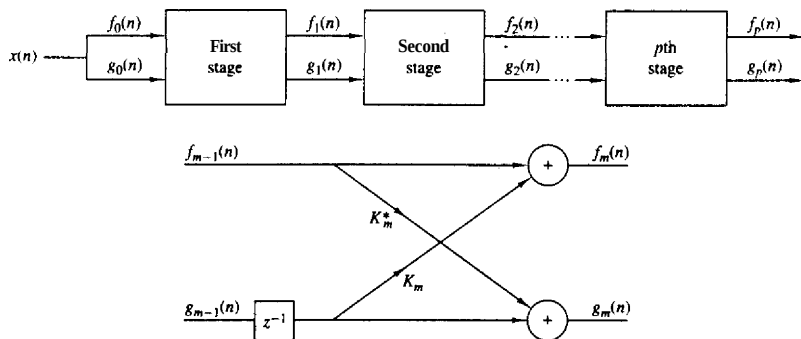


Figure 11.4 p-stage lattice filter

$\mathcal{E}_p^f$ is a quadratic function of the predictor coefficients and its minimization leads to the set of linear equations

$$\gamma_{xx}(l) = - \sum_{k=1}^p a_p(k) \gamma_{xx}(l-k) \quad l = 1, 2, \dots, p \quad (11.2.9)$$

These are called the *normal equations* for the coefficients of the linear predictor. The minimum mean-square prediction error is simply

$$\min[\mathcal{E}_p^f] \equiv E_p^f = \gamma_{xx}(0) + \sum_{k=1}^p a_p(k) \gamma_{xx}(-k) \quad (11.2.10)$$

In the following section we extend the development above to the problem of predicting the value of a time series in the opposite direction, namely, backward in time.

11.2.2 Backward Linear Prediction

Let us assume that we have the data sequence $x(n), x(n-1), \dots, x(n-p+1)$ from a stationary random process and we wish to predict the value $x(n-p)$ of the process. In this case we employ a one-step *backward linear predictor* of order p . Hence

$$\hat{x}(n-p) = - \sum_{k=0}^{p-1} b_p(k) x(n-k) \quad (11.2.11)$$

The difference between the value $x(n-p)$ and the estimate $\hat{x}(n-p)$ is called the *backward prediction error*, denoted as $g_p(n)$:

$$\begin{aligned} g_p(n) &= x(n-p) + \sum_{k=0}^{p-1} b_p(k) x(n-k) \\ &= \sum_{k=0}^p b_p(k) x(n-k) \quad b_p(p) = 1 \end{aligned} \quad (11.2.12)$$

The backward linear predictor can be realized either by a direct-form FIR filter structure similar to the structure shown in Fig. 11.2 or as a lattice structure. The lattice structure shown in Fig. 11.4 provides the backward linear predictor as well as the forward linear predictor.

The weighting coefficients in the backward linear predictor are the complex conjugates of the coefficients for the forward linear predictor, but they occur in reverse order. Thus we have

$$b_p(k) = a_p^*(p-k) \quad k = 0, 1, \dots, p \quad (11.2.13)$$

In the z -domain, the convolution sum in (11.2.12) becomes

$$G_p(z) = B_p(z) X(z) \quad (11.2.14)$$

or, equivalently,

$$B_p(z) = \frac{G_p(z)}{X(z)} = \frac{G_p(z)}{G_0(z)} \quad (11.2.15)$$

where $B_p(z)$ represents the system function of the FIR filter with coefficients $b_p(k)$.

Since $b_p(k) = a^*(p-k)$, $G_p(z)$ is related to $A_p(z)$

$$\begin{aligned} B_p(z) &= \sum_{k=0}^p b_p(k)z^{-k} \\ &= \sum_{k=0}^p a_p^*(p-k)z^{-k} \\ &= z^{-p} \sum_{k=0}^p a_p^*(k)z^k \\ &= z^{-p} A_p^*(z^{-1}) \end{aligned} \quad (11.2.16)$$

The relationship in (11.2.16) implies that the zeros of the FIR filter with system function $B_p(z)$ are simply the (conjugate) reciprocals of the zeros of $A_p(z)$. Hence $B_p(z)$ is called the reciprocal or **reverse polynomial** of $A_p(z)$.

Now that we have established these interesting relationships between the forward and backward one-step predictors, let us return to the recursive lattice equations in (11.2.4) and transform them to the z -domain. Thus we have

$$\begin{aligned} F_0(z) &= G_0(z) = X(z) \\ F_m(z) &= F_{m-1}(z) + K_m z^{-1} G_{m-1}(z) \quad m = 1, 2, \dots, p \\ G_m(z) &= K_m^* F_{m-1}(z) + z^{-1} G_{m-1}(z) \quad m = 1, 2, \dots, p \end{aligned} \quad (11.2.17)$$

If we divide each equation by $X(z)$, we obtain the desired results in the form

$$\begin{aligned} A_0(z) &= B_0(z) = 1 \\ A_m(z) &= A_{m-1}(z) + K_m z^{-1} B_{m-1}(z) \quad m = 1, 2, \dots, p \\ B_m(z) &= K_m^* A_{m-1}(z) + z^{-1} B_{m-1}(z) \quad m = 1, 2, \dots, p \end{aligned} \quad (11.2.18)$$

Thus a lattice filter is described in the z -domain by the matrix equation

$$\begin{bmatrix} A_m(z) \\ B_m(z) \end{bmatrix} = \begin{bmatrix} 1 & K_m z^{-1} \\ K_m^* & z^{-1} \end{bmatrix} \begin{bmatrix} A_{m-1}(z) \\ B_{m-1}(z) \end{bmatrix} \quad (11.2.19)$$

The relations in (11.2.17) for $A_m(z)$ and $B_m(z)$ allow us to obtain the direct-form FIR filter coefficients $\{a_m(k)\}$ from the reflection coefficients $\{K_m\}$, and vice versa. These relationships were given in Section 7.2.4 by (7.2.51) through (7.2.53).

The **lattice** structure with parameters $K_1, K_2, \dots, K_p$ corresponds to a class of p direct-form FIR filters with system functions $A_1(z), A_2(z), \dots, A_p(z)$. It is interesting to note that a characterization of this class of p FIR filters in direct form requires $p(p+1)/2$ filter coefficients. In contrast, the lattice-form characterization requires only the p reflection coefficients $\{K_i\}$. The reason the lattice provides a more compact representation for the class of p FIR filters is because appending

stages to the lattice does not alter the parameters of the previous stages. On the other hand, appending the p th stage to a lattice with $(p-1)$ stages is equivalent to increasing the length of an FIR filter by one coefficient. The resulting FIR filter with system function $A_p(z)$ has coefficients totally different from the coefficients of the lower-order FIR filter with system function $A_{p-1}(z)$.

The formula for determining the filter coefficients $\{a_p(k)\}$ recursively is easily derived from polynomial relationships (11.2.18). We have

$$A_m(z) = A_{m-1}(z) + K_m z^{-1} B_{m-1}(z)$$

$$\sum_{k=0}^m a_m(k) z^{-k} = \sum_{k=0}^{m-1} a_{m-1}(k) z^{-k} + K_m \sum_{k=0}^{m-1} a_{m-1}^*(m-1-k) z^{-(k+1)} \quad (11.2.20)$$

By equating the coefficients of equal powers of z^{-1} and recalling that $a_m(0) = 1$ for $m = 1, 2, \dots, p$, we obtain the desired recursive equation for the FIR filter coefficients in the form

$$a_m(0) = 1$$

$$a_m(m) = K_m$$

$$a_m(k) = a_{m-1}(k) + K_m a_{m-1}^*(m-k) \quad (11.2.21)$$

$$= a_{m-1}(k) + a_m(m) a_{m-1}^*(m-k) \quad 1 \leq k \leq m-1$$

$$m = 1, 2, \dots, p$$

The conversion formula from the direct-form FIR filter coefficients $\{a_p(k)\}$ to the lattice reflection coefficients $\{K_i\}$ is also very simple. For the p -stage lattice we immediately obtain the reflection coefficient $K_p = a_p(p)$. To obtain $K_{p-1}, \dots, K_1$, we need the polynomials $A_m(z)$ for $m = p-1, \dots, 1$. From (11.2.19) we obtain

$$A_{m-1}(z) = \frac{A_m(z) - K_m B_m(z)}{1 - |K_m|^2} \quad m = p, \dots, 1 \quad (11.2.22)$$

which is just a step-down recursion. Thus we compute all lower-degree polynomials $A_m(z)$ beginning with $A_{p-1}(z)$ and obtain the desired lattice reflection coefficients from the relation $K_m = a_m(m)$. We observe that the procedure works as long as $|K_m| \neq 1$ for $m = 1, 2, \dots, p-1$. From this step-down recursion for the polynomials, it is relatively easy to obtain a formula for recursively and directly computing K_m , $m = p-1, \dots, 1$. For $m = p-1, \dots, 1$, we have

$$K_m = a_m(m)$$

$$a_{m-1}(k) = \frac{a_m(k) - K_m b_m(k)}{1 - |K_m|^2} \quad (11.2.23)$$

$$= \frac{a_m(k) - a_m(m) a_m^*(m-k)}{1 - |a_m(m)|^2}$$

which is just the recursion in the **Schür-Cohn** stability test for the polynomial $A_m(z)$.

As just indicated, the recursive equation in (11.2.23) breaks down if any of the lattice parameters $|K_m| = 1$. If this occurs, it is indicative that the polynomial $A_{m-1}(z)$ has a root located on the unit circle. Such a root can be factored out from $A_{m-1}(z)$ and the iterative process in (11.2.23) carried out for the reduced-order system.

Finally, let us consider the **minimization** of the **mean-square** error in a backward linear predictor. The backward prediction error is

$$\begin{aligned} g_p(n) &= x(n-p) + \sum_{k=0}^{p-1} b_p(k)x(n-k) \\ &= x(n-p) + \sum_{k=1}^p a_p^*(k)x(n-p+k) \end{aligned} \quad (11.2.24)$$

and its mean-square value is

$$\mathcal{E}_p^b = E[|g_p(n)|^2] \quad (11.2.25)$$

The minimization of $\mathcal{E}_p^b$ with respect to the prediction coefficients yields the same set of linear equations as in (11.2.9). Hence the minimum mean-square error is

$$\min[\mathcal{E}_p^b] \equiv E_p^b = E_p^f \quad (11.2.26)$$

which is given by (11.2.10).

11.2.3 The Optimum Reflection Coefficients for the Lattice Forward and Backward Predictors

In Sections 11.2.1 and 11.2.2 we derived the set of linear equations which provide the predictor coefficients that minimize the mean-square value of the prediction error. In this section we consider the problem of optimizing the reflection coefficients in the lattice predictor and expressing the reflection coefficients in terms of the **forward** and backward prediction errors.

The forward prediction error in the lattice **filter** is expressed as

$$f_m(n) = f_{m-1}(n) + K_m g_{m-1}(n-1) \quad (11.2.27)$$

The minimization of $E[|f_m(n)|^2]$ with respect to the reflection coefficient K_m yields the result

$$K_m = \frac{-E[f_{m-1}(n)g_{m-1}^*(n-1)]}{E[|g_{m-1}(n-1)|^2]} \quad (11.2.28)$$

or, equivalently,

$$K_m = \frac{-E[f_{m-1}(n)g_{m-1}^*(n-1)]}{\sqrt{E_{m-1}^f E_{m-1}^b}} \quad (11.2.29)$$

where $E_{m-1}^f = E_{m-1}^b = E[|g_{m-1}(n-1)|^2] = E[|f_{m-1}(n)|^2]$.

We observe that the optimum choice of the reflection coefficients in the lattice predictor is the negative of the (**normalized**) crosscorrelation coefficients

between the forward and backward errors in the lattice.* Since it is apparent from (11.2.28) that $|K_m| \leq 1$, it follows that the minimum mean-square value of the prediction error, which can be expressed recursively as

$$E_m^f = (1 - |K_m|^2) E_{m-1}^f \quad (11.2.30)$$

is a monotonically decreasing sequence.

11.2.4 Relationship of an AR Process to Linear Prediction

The parameters of an $\text{AR}(p)$ process are intimately related to a predictor of order p for the same process. To see the relationship, we recall that in an $\text{AR}(p)$ process, the autocorrelation sequence $\{\gamma_{xx}(m)\}$ is related to the parameters $\{a_k\}$ by the Yule-Walker equations given in (11.1.19) or (11.1.20). The corresponding equations for the predictor of order p are given by (11.2.9) and (11.2.10).

A direct comparison of these two sets of relations reveals that there is a one-to-one correspondence between the parameters $\{a_k\}$ of the $\text{AR}(p)$ process and the predictor coefficients $\{a_p(k)\}$ of the p th-order predictor. In fact, if the underlying process $\{x(n)\}$ is $\text{AR}(p)$, the prediction coefficients of the p th-order predictor are identical to $\{a_k\}$. Furthermore, the minimum MSE in the p th-order predictor E_p^f is identical to σ_w^2 , the variance of the white noise process. In this case, the prediction-error filter is a noise-whitening filter which produces the innovations sequence $\{w(n)\}$.

11.3 SOLUTION OF THE NORMAL EQUATIONS

In the preceding section we observed that the minimization of the mean-square value of the forward prediction error resulted in a set of linear equations for the coefficients of the predictor given by (11.2.9). These equations, called the normal equations, may be expressed in the compact form

$$\sum_{k=0}^p a_p(k) \gamma_{xx}(l-k) = 0 \quad \begin{matrix} l = 1, 2, \dots, p \\ a_p(0) = 1 \end{matrix} \quad (11.3.1)$$

The resulting minimum **MSE (MMSE)** is given by (11.2.10). If we augment (11.2.10) to the normal equations given by (11.3.1) we obtain the set of **augmented normal equations**, which may be expressed as

$$\sum_{k=0}^p a_p(k) \gamma_{xx}(l-k) = \begin{cases} E_p^f, & l = 0 \\ 0, & l = 1, 2, \dots, p \end{cases} \quad (11.3.2)$$

We also noted that if the random process is an $\text{AR}(p)$ process, the **MMSE** $E_p^f = \sigma_w^2$.

*The normalized crosscorrelation coefficients between the forward and backward error in the lattice (i.e., $\{-K_m\}$) are often called the partial correlation (PARCOR) coefficients.

In this section we describe two computationally efficient algorithms for solving the normal equations. One algorithm, originally due to **Levinson (1947)** and modified by **Durbin (1959)**, is called the Levinson–Durbin algorithm. This algorithm is suitable for serial processing and has a computation complexity of $O(p^2)$. The second algorithm, due to **Schir (1917)**, also computes the reflection coefficients in $O(p^2)$ operations but with parallel processors, the computations can be performed in $O(p)$ time. Both algorithms exploit the Toeplitz symmetry property inherent in the autocorrelation matrix.

We begin by describing the Levinson–Durbin algorithm.

11.3.1 The Levinson–Durbin Algorithm

The Levinson–Durbin algorithm is a computationally efficient algorithm for solving the normal equations in **(11.3.1)** for the prediction coefficients. This algorithm exploits the special symmetry in the autocorrelation matrix

$$r = \begin{bmatrix} \gamma_{xx}(0) & \gamma_{xx}^*(1) & \cdots & \gamma_{xx}^*(p-1) \\ \gamma_{xx}(1) & \gamma_{xx}(0) & \cdots & \gamma_{xx}^*(p-2) \\ \vdots & \vdots & \ddots & \vdots \\ \gamma_{xx}(p-1) & \gamma_{xx}(p-2) & \cdots & \gamma_{xx}(0) \end{bmatrix} \quad (11.3.3)$$

Note that $\Gamma_p(i, j) = \Gamma_p(i - j)$, so that the autocorrelation matrix is a *Toeplitz* matrix. Since $\Gamma_p(i, j) = \Gamma_p^*(j, i)$, the matrix is also Hermitian.

The key to the Levinson–Durbin method of solution, that exploits the Toeplitz property of the matrix, is to proceed recursively, beginning with a predictor of order $m = 1$ (one coefficient) and then to increase the order recursively, using the lower-order solutions to obtain the solution to the next-higher order. Thus the solution to the first-order predictor obtained by solving **(11.3.1)** is

$$a_1(1) = -\frac{\gamma_{xx}(1)}{\gamma_{xx}(0)} \quad (11.3.4)$$

and the resulting MMSE is

$$\begin{aligned} E_1^f &= \gamma_{xx}(0) + a_1(1)\gamma_{xx}(-1) \\ &= \gamma_{xx}(0)[1 - |a_1(1)|^2] \end{aligned} \quad (11.3.5)$$

Recall that $a_1(1) = K_1$, the first reflection coefficient in the lattice filter.

The next step is to solve for the coefficients $\{a_2(1), a_2(2)\}$ of the second-order predictor and express the solution in terms of $a_1(1)$. The two equations obtained from **(11.3.1)** are

$$\begin{aligned} a_2(1)\gamma_{xx}(0) + a_2(2)\gamma_{xx}^*(1) &= -\gamma_{xx}(1) \\ a_2(1)\gamma_{xx}(1) + a_2(2)\gamma_{xx}(0) &= -\gamma_{xx}(2) \end{aligned} \quad (11.3.6)$$

By using the solution in (11.3.4) to eliminate $\gamma_{xx}(1)$, we obtain the solution

$$\begin{aligned} a_2(2) &= -\frac{\gamma_{xx}(2) + a_1(1)\gamma_{xx}(1)}{\gamma_{xx}(0)[1 - |a_1(1)|^2]} \\ &= -\frac{\gamma_{xx}(2) + a_1(1)\gamma_{xx}(1)}{E_1^f} \quad (11.3.7) \\ a_2(1) &= a_1(1) + a_2(2)a_1^*(1) \end{aligned}$$

Thus we have obtained the coefficients of the second-order predictor. Again, we note that $a_2(2) = K_2$, the second reflection coefficient in the lattice filter.

Proceeding in this manner, we can express the coefficients of the m th-order predictor in terms of the coefficients of the $(m-1)$ st-order predictor. Thus we can write the coefficient vector $\mathbf{a}_m$ as the sum of two vectors, namely,

$$\mathbf{a}_m = \begin{bmatrix} a_m(1) \\ a_m(2) \\ \vdots \\ a_m(m) \end{bmatrix} = \begin{bmatrix} \mathbf{a}_{m-1} \\ \vdots \\ 0 \end{bmatrix} + \begin{bmatrix} \mathbf{d}_{m-1} \\ \vdots \\ K_m \end{bmatrix} \quad (11.3.8)$$

where $\mathbf{a}_{m-1}$ is the predictor coefficient vector of the $(m-1)$ st-order predictor and the vector $\mathbf{d}_{m-1}$ and the scalar K_m are to be determined. Let us also partition the $m \times m$ autocorrelation matrix Γ_{xx} as

$$\Gamma_m = \begin{bmatrix} \Gamma_{m-1} & \boldsymbol{\gamma}_{m-1}^{b*} \\ \boldsymbol{\gamma}_{m-1}^{bt} & \gamma_{xx}(0) \end{bmatrix} \quad (11.3.9)$$

where $\boldsymbol{\gamma}_{m-1}^{bt} = [\gamma_{xx}(m-1) \ \gamma_{xx}(m-2) \ \cdots \ \gamma_{xx}(1)] = (\boldsymbol{\gamma}_{m-1}^b)^t$, the asterisk (*) denotes the complex conjugate, and $\boldsymbol{\gamma}_m^t$ denotes the transpose of $\boldsymbol{\gamma}_m$. The superscript b on $\boldsymbol{\gamma}_{m-1}$ denotes the vector $\boldsymbol{\gamma}_{m-1}^b = [\gamma_{xx}(1) \ \gamma_{xx}(2) \ \cdots \ \gamma_{xx}(m-1)]$ with elements taken in reverse order.

The solution to the equation $\Gamma_m \mathbf{a}_m = -\boldsymbol{\gamma}_m$ can be expressed as

$$\begin{bmatrix} \Gamma_{m-1} & \boldsymbol{\gamma}_{m-1}^{b*} \\ \boldsymbol{\gamma}_{m-1}^{bt} & \gamma_{xx}(0) \end{bmatrix} \left\{ \begin{bmatrix} \mathbf{a}_{m-1} \\ 0 \end{bmatrix} + \begin{bmatrix} \mathbf{d}_{m-1} \\ K_m \end{bmatrix} \right\} = -\begin{bmatrix} \boldsymbol{\gamma}_{m-1} \\ \gamma_{xx}(m) \end{bmatrix} \quad (11.3.10)$$

This is the key step in the Levinson–Durbin algorithm. From (11.3.10) we obtain two equations, namely,

$$\Gamma_{m-1} \mathbf{a}_{m-1} + \Gamma_{m-1} \mathbf{d}_{m-1} + K_m \boldsymbol{\gamma}_{m-1}^{b*} = -\boldsymbol{\gamma}_{m-1} \quad (11.3.11)$$

$$\boldsymbol{\gamma}_{m-1}^{bt} \mathbf{a}_{m-1} + \boldsymbol{\gamma}_{m-1}^{bt} \mathbf{d}_{m-1} + K_m \gamma_{xx}(0) = -\gamma_{xx}(m) \quad (11.3.12)$$

Since $\Gamma_{m-1} \mathbf{a}_{m-1} = -\boldsymbol{\gamma}_{m-1}$, (11.3.11) yields the solution

$$\mathbf{d}_{m-1} = -K_m \Gamma_{m-1}^{-1} \boldsymbol{\gamma}_{m-1}^{b*} \quad (11.3.13)$$

But $\boldsymbol{\gamma}_{m-1}^{b*}$ is just $\boldsymbol{\gamma}_{m-1}$ with elements taken in reverse order and conjugated. There-

fore, the solution in (11.3.13) is simply

$$\mathbf{d}_{m-1} = K_m \mathbf{a}_{m-1}^{b^*} = K_m \begin{bmatrix} a_{m-1}^*(m-1) \\ a_{m-1}^*(m-2) \\ \vdots \\ a_{m-1}^*(1) \end{bmatrix} \quad (11.3.14)$$

The scalar equation (11.3.12) can now be used to solve for K_m . If we eliminate $\mathbf{d}_{m-1}$ in (11.3.12) by using (11.3.14), we obtain

$$K_m [\gamma_{xx}(0) + \gamma_{m-1}^{bt} \mathbf{a}_{m-1}^{b^*}] + \gamma_{m-1}^{bt} \mathbf{a}_{m-1} = -\gamma_{xx}(m)$$

Hence

$$K_m = -\frac{\gamma_{xx}(m) + \gamma_{m-1}^{bt} \mathbf{a}_{m-1}}{\gamma_{xx}(0) + \gamma_{m-1}^{bt} \mathbf{a}_{m-1}^{b^*}} \quad (11.3.15)$$

Therefore, by substituting the solutions in (11.3.14) and (11.3.15) into (11.3.8), we obtain the desired recursion for the predictor coefficients in the Levinson-Durbin algorithm as

$$a_m(m) = K_m = -\frac{\gamma_{xx}(m) + \gamma_{m-1}^{bt} \mathbf{a}_{m-1}}{\gamma_{xx}(0) + \gamma_{m-1}^{bt} \mathbf{a}_{m-1}^{b^*}} = -\frac{\gamma_{xx}(m) + \gamma_{m-1}^{bt} \mathbf{a}_{m-1}}{E_m^f} \quad (11.3.16)$$

$$\begin{aligned} a_m(k) &= a_{m-1}(k) + K_m a_{m-1}^*(m-k) \\ &= a_{m-1}(k) + a_m(m) a_{m-1}^*(m-k) \quad \begin{matrix} k = 1, 2, \dots, m-1 \\ m = 1, 2, \dots, p \end{matrix} \end{aligned} \quad (11.3.17)$$

The reader should note that the recursive relation in (11.3.17) is identical to the recursive relation in (11.2.21) for the predictor coefficients, obtained from the polynomials $A_m(z)$ and $B_m(z)$. Furthermore, K_m is the reflection coefficient in the m th stage of the lattice predictor. This development clearly illustrates that the Levinson-Durbin algorithm produces the reflection coefficients for the optimum lattice prediction filter as well as the coefficients of the optimum direct-form FIR predictor.

Finally, let us determine the expression for the MMSE. For the m th-order predictor, we have

$$\begin{aligned} E_m^f &= \gamma_{xx}(0) + \sum_{k=1}^m a_m(k) \gamma_{xx}(-k) \\ &= \gamma_{xx}(0) + \sum_{k=1}^m [a_{m-1}(k) + a_m(m) a_{m-1}^*(m-k)] \gamma_{xx}(-k) \\ &= E_{m-1}^f [1 - |a_m(m)|^2] = E_{m-1}^f (1 - |K_m|^2) \quad m = 1, 2, \dots, p \end{aligned} \quad (11.3.18)$$

where $E_0^f = \gamma_{xx}(0)$. Since the reflection coefficients satisfy the property that $|K_m| \leq 1$, the MMSE for the sequence of predictors satisfies the condition

$$E_0^f \geq E_1^f \geq E_2^f \geq \dots \geq E_p^f \quad (11.3.19)$$

This concludes the derivation of the Levinson–Durbin algorithm for solving the linear equations $\Gamma_m \mathbf{a}_m = -\gamma_m$, for $m = 0, 1, \dots, p$. We observe that the linear equations have the special property that the vector on the right-hand side also appears as a vector in Γ_m . In the more general case, where the vector on the right-hand side is some other vector, say $\mathbf{c}_m$, the set of linear equations can be solved recursively by introducing a second recursive equation to solve the more general linear equations $\Gamma_m \mathbf{b}_m = \mathbf{c}_m$. The result is a generalized Levinson–Durbin algorithm (see Problem 11.12).

The Levinson–Durbin recursion given by (11.3.17) requires $O(m)$ multiplications and additions (operations) to go from stage m to stage $m+1$. Therefore, for p stages it takes on the order of $1+2+3+\dots+p(p+1)/2$, or $O(p^2)$, operations to solve for the prediction filter coefficients, or the reflection coefficients, compared with $O(p^3)$ operations if we did not exploit the Toeplitz property of the correlation matrix.

If the Levinson–Durbin algorithm is implemented on a serial computer or signal processor, the required computation time is on the order of $O(p^2)$ time units. On the other hand, if the processing is performed on a parallel processing machine utilizing as many processors as necessary to exploit the full parallelism in the algorithm, the multiplications as well as the additions required to compute (11.3.17) can be carried out **simultaneously**. Therefore, this computation can be **performed** in $O(p)$ time units. However, the computation in (11.3.16) for the reflection coefficients takes additional time. Certainly, the inner products involving the vectors $\mathbf{a}_{m-1}$ and $\mathbf{y}_{m-1}^p$ can be computed **simultaneously** by employing parallel processors. However, the addition of these products cannot be done simultaneously, but instead, require $O(\log p)$ time units. Hence, the computations in the Levinson–Durbin algorithm, when performed by p parallel processors, can be accomplished in $O(p \log p)$ time.

In the following section we describe another algorithm, due to Schiir (1917), that avoids the computation of inner products, and therefore is more suitable for parallel computation of the reflection coefficients.

11.3.2 The Schiir Algorithm

The Schiir algorithm is intimately related to a recursive test for determining the positive definiteness of a correlation matrix. To be specific, let us consider the **autocorrelation** matrix Γ_{p+1} associated with the augmented normal equations given by (11.3.2). From the elements of this matrix we form the function

$$R_0(z) = \frac{\gamma_{xx}(1)z^{-1} + \gamma_{xx}(2)z^{-2} + \dots + \gamma_{xx}(p)z^{-p}}{\gamma_{xx}(0) + \gamma_{xx}(1)z^{-1} + \dots + \gamma_{xx}(p)z^{-p}} \quad (11.3.20)$$

and the sequence of functions $R_m(z)$ defined recursively as

$$R_m(z) = \frac{R_{m-1}(z) - R_{m-1}(\infty)}{z^{-1}[1 - R_{m-1}^*(\infty)R_{m-1}(z)]} \quad m = 1, 2, \dots \quad (11.3.21)$$

Schür's theorem states that a necessary and sufficient condition for the correlation matrix to be positive definite is that $|R_m(\infty)| < 1$ for $m = 1, 2, \dots, p$.

Let us demonstrate that the condition for positive definiteness of the autocorrelation matrix Γ_{p+1} is equivalent to the condition that the reflection coefficients in the equivalent lattice filter satisfy the condition $|K_m| < 1$, $m = 1, 2, \dots, p$.

First, we note that $R_0(\infty) = 0$. Then, from (11.3.21) we have

$$R_1(z) = \frac{\gamma_{xx}(1) + \gamma_{xx}(2)z^{-1} + \dots + \gamma_{xx}(p)z^{-p+1}}{\gamma_{xx}(0) + \gamma_{xx}(1)z^{-1} + \dots + \gamma_{xx}(p)z^{-p}} \quad (11.3.22)$$

Hence $R_1(\infty) = \gamma_{xx}(1)/\gamma_{xx}(0)$. We observe that $R_1(\infty) = -K_1$.

Second, we compute $R_2(z)$ according to (11.3.21) and evaluate the result at $z = \infty$. Thus we obtain

$$R_2(\infty) = \frac{\gamma_{xx}(2) + K_1\gamma_{xx}(1)}{\gamma_{xx}(0)(1 - |K_1|^2)}$$

Again, we observe that $R_2(\infty) = -K_2$. By continuing this procedure, we find that $R_m(\infty) = -K_m$, for $m = 1, 2, \dots, p$. Hence the condition $|R_m(\infty)| < 1$ for $m = 1, 2, \dots, p$, is identical to the condition $|K_m| < 1$ for $m = 1, 2, \dots, p$, and ensures the positive definiteness of the autocorrelation matrix Γ_{p+1} .

Since the reflection coefficients can be obtained from the sequence of functions $R_m(z)$, $m = 1, 2, \dots, p$, we have another method for solving the normal equations. We call this method the Schür algorithm.

Schür algorithm. Let us first rewrite $R_m(z)$ as

$$R_m(z) = \frac{P_m(z)}{Q_m(z)} \quad m = 0, 1, \dots, p \quad (11.3.23)$$

where

$$\begin{aligned} P_0(z) &= \gamma_{xx}(1)z^{-1} + \gamma_{xx}(2)z^{-2} + \dots + \gamma_{xx}(p)z^{-p} \\ Q_0(z) &= \gamma_{xx}(0) + \gamma_{xx}(1)z^{-1} + \dots + \gamma_{xx}(p)z^{-p} \end{aligned} \quad (11.3.24)$$

Since $K_0 = 0$ and $K_m = -R_m(\infty)$ for $m = 1, 2, \dots, p$, the recursive equation (11.3.21) implies the following recursive equations for the polynomials $P_m(z)$ and $Q_m(z)$:

$$\begin{bmatrix} P_m(z) \\ Q_m(z) \end{bmatrix} = \begin{bmatrix} 1 & K_{m-1} \\ K_{m-1}^* z^{-1} & z^{-1} \end{bmatrix} \begin{bmatrix} P_{m-1}(z) \\ Q_{m-1}(z) \end{bmatrix} \quad m = 1, 2, \dots, p \quad (11.3.25)$$

Thus we have

$$\begin{aligned} P_1(z) &= P_0(z) = \gamma_{xx}(1)z^{-1} + \gamma_{xx}(2)z^{-2} + \dots + \gamma_{xx}(p)z^{-p} \\ Q_1(z) &= z^{-1}Q_0(z) = \gamma_{xx}(0)z^{-1} + \gamma_{xx}(1)z^{-2} + \dots + \gamma_{xx}(p)z^{-p-1} \end{aligned} \quad (11.3.26)$$

and

$$K_1 = - \left. \frac{P_1(z)}{Q_1(z)} \right|_{z=\infty} = - \frac{\gamma_{xx}(1)}{\gamma_{xx}(0)} \quad (11.3.27)$$

Next the reflection coefficient K_2 is obtained by determining $P_2(z)$ and $Q_2(z)$ from (11.3.25), dividing $P_2(z)$ by $Q_2(z)$ and evaluating the result at $z = \infty$. Thus we find that

$$\begin{aligned} P_2(z) &= P_1(z) + K_1 Q_1(z) \\ &= [\gamma_{xx}(2) + K_1 \gamma_{xx}(1)]z^{-2} + \cdots \\ &\quad + [\gamma_{xx}(p) + K_1 \gamma_{xx}(p-1)]z^{-p} \\ Q_2(z) &= z^{-1}[Q_1(z) + K_1^* P_1(z)] \\ &= [\gamma_{xx}(0) + K_1^* \gamma_{xx}(1)]z^{-2} + \cdots \\ &\quad + [\gamma_{xx}(p-2) + K_1^* \gamma_{xx}(p-1)]z^{-p} \end{aligned} \quad (11.3.28)$$

where the terms involving z^{-p-1} have been dropped. Thus we observe that the recursive equation in (11.3.25) is equivalent to (11.3.21).

Based on these relationships, the Schür algorithm is described by the following recursive procedure.

Initialization. Form the $2x(p+1)$ generator matrix

$$\mathbf{G}_0 = \begin{bmatrix} 0 & \gamma_{xx}(1) & \gamma_{xx}(2) & \cdots & \gamma_{xx}(p) \\ \gamma_{xx}(0) & \gamma_{xx}(1) & \gamma_{xx}(2) & \cdots & \gamma_{xx}(p) \end{bmatrix} \quad (11.3.29)$$

where the elements of the first row are the coefficients of $P_0(z)$ and the elements of the second row are the coefficients of $Q_0(z)$.

Step 1. Shift the second row of the generator matrix to the right by one place and discard the last element of this row. A zero is placed in the vacant position. Thus we obtain a new generator matrix,

$$\mathbf{G}_1 = \begin{bmatrix} 0 & \gamma_{xx}(1) & \gamma_{xx}(2) & \cdots & \gamma_{xx}(p) \\ 0 & \gamma_{xx}(0) & \gamma_{xx}(1) & \cdots & \gamma_{xx}(p-1) \end{bmatrix} \quad (11.3.30)$$

The (negative) ratio of the elements in the second column yield the reflection coefficient $K_1 = -\gamma_{xx}(1)/\gamma_{xx}(0)$.

Step 2. Multiply the generator matrix by the 2×2 matrix

$$\mathbf{V}_1 = \begin{bmatrix} 1 & K_1 \\ K_1^* & 1 \end{bmatrix} \quad (11.3.31)$$

Thus we obtain

$$\mathbf{V}_1 \mathbf{G}_1 = \begin{bmatrix} 0 & 0 & \gamma_{xx}(2) + K_1 \gamma_{xx}(1) & \cdots & \gamma_{xx}(p) + K_1 \gamma_{xx}(p-1) \\ 0 & \gamma_{xx}(0) + K_1^* \gamma_{xx}(1) & \cdots & \cdots & \gamma_{xx}(p-1) + K_1^* \gamma_{xx}(p) \end{bmatrix} \quad (11.3.32)$$

Step 3. Shift the second row of $\mathbf{V}_1 \mathbf{G}_1$ by one place to the right and thus form the new generator matrix

$$\mathbf{G}_2 = \begin{bmatrix} 0 & 0 & \gamma_{xx}(2) + K_1 \gamma_{xx}(1) & \cdots & \gamma_{xx}(p) + K_1 \gamma_{xx}(p-1) \\ 0 & 0 & \gamma_{xx}(0) + K_1^* \gamma_{xx}(1) & \cdots & \gamma_{xx}(p-2) + K_1^* \gamma_{xx}(p-1) \end{bmatrix} \quad (11.3.33)$$

The negative ratio of the elements in the third column of $\mathbf{G}_2$ yields K_2 .

Steps 2 and 3 are repeated until we have solved for all p reflection coefficients. In general, the 2×2 matrix in step 2 is

$$\mathbf{V}_m = \begin{bmatrix} 1 & K_m \\ K_m^* & 1 \end{bmatrix} \quad (11.3.34)$$

and multiplication of $\mathbf{V}_m$ by $\mathbf{G}_m$ yields $\mathbf{V}_m \mathbf{G}_m$. In step 3 we shift the second row of $\mathbf{V}_m \mathbf{G}_m$ one place to the right and obtain the new generator matrix $\mathbf{G}_{m+1}$.

We observe that the shifting operation of the second row in each iteration is equivalent to multiplication by the delay operator z^{-1} in the second recursive equation in (11.3.25). We also note that the division of the polynomial $P_m(z)$ by the polynomial $Q_m(z)$ and the evaluation of the quotient at $z = \infty$ is equivalent to dividing the elements in the $(m+1)$ st column of $\mathbf{G}$. The computation of the p reflection coefficients can be accomplished by use of parallel processors in $O(p)$ time units. Below we describe a pipelined architecture for performing these computations.

Another way of demonstrating the relationship of the Schur algorithm to the Levinson-Durbin algorithm and the corresponding lattice predictor is to determine the output of the lattice filter obtained when the input sequence is the correlation sequence $\{\gamma_{xx}(m), m = 0, 1, \dots\}$. Thus, the first input to the lattice filter is $\gamma_{xx}(0)$, the second input is $\gamma_{xx}(1)$, and so on [i.e., $f_0(n) = \gamma_{xx}(n)$]. After the delay in the first stage, we have $g_0(n-1) = \gamma_{xx}(n-1)$. Hence, for $n = 1$, the ratio $f_0(1)/g_0(0) = \gamma_{xx}(1)/\gamma_{xx}(0)$, which is the negative of the reflection coefficient K_1 . Alternatively, we can express this relationship as

$$f_0(1) + K_1 g_0(0) = \gamma_{xx}(1) + K_1 \gamma_{xx}(0) = 0$$

Furthermore, $g_0(0) = \gamma_{xx}(0) = E_0^f$. At time $n = 2$, the input to the second stage is, according to (11.2.4),

$$f_1(2) = f_0(2) + K_1 g_0(1) = \gamma_{xx}(2) + K_1 \gamma_{xx}(1)$$

and after the unit of delay in the second stage, we have

$$g_1(1) = K_1^* f_0(1) + g_0(0) = K_1^* \gamma_{xx}(1) + \gamma_{xx}(0)$$

Now the ratio $f_1(2)/g_1(1)$ is

$$\frac{f_1(2)}{g_1(1)} = \frac{\gamma_{xx}(2) + K_1 \gamma_{xx}(1)}{\gamma_{xx}(0) + K_1^* \gamma_{xx}(1)} = \frac{\gamma_{xx}(2) + K_1 \gamma_{xx}(1)}{E_1^f} = -K_2$$

Hence

$$f_1(2) + K_2 g_1(1) = 0$$

$$g_1(1) = E_1^f$$

By continuing in this manner, we can show that at the input to the m th lattice stage, the ratio $f_{m-1}(m)/g_{m-1}(m-1) = -K_m$ and $g_{m-1}(m-1) = E_{m-1}^f$. Consequently, the lattice filter coefficients obtained from the Levinson algorithm are identical to the coefficients obtained in the Schür algorithm. Furthermore, the lattice filter

structure provides a method for computing the reflection coefficients in the lattice predictor.

A pipelined architecture for implementing the Schür algorithm. Kung and Hu (1983) developed a pipelined lattice-type processor for implementing the Schür algorithm. The processor consists of a cascade of p lattice-type stages, where each stage consists of two processing elements (PEs), which we designate as upper PEs denoted as $A_1, A_2, \dots, A_p$, and lower PEs denoted as $B_1, B_2, \dots, B_p$, as shown in Fig. 11.5. The PE designated as A_1 is assigned the task of performing divisions. The remaining PEs perform one multiplication and one addition per iteration (one clock cycle).

Initially, the upper PEs are loaded with the elements of the first row of the generator matrix G_0 , as illustrated in Fig. 11.5. The lower PEs are loaded with the elements of the second row of the generator matrix G_0 . The computational process begins with the division PE, A_1 , which computes the first reflection coefficient as $K_1 = -\gamma_{xx}(1)/\gamma_{xx}(0)$. The value of K_1 is sent simultaneously to all the PEs in the upper branch and lower branch.

The second step in the computation is to update the contents of all processing elements simultaneously. The contents of the upper and lower PEs are updated as follows:

$$\begin{aligned} \text{PE } A_m: A_m &\leftarrow A_m + K_1 B_m & m = 2, 3, \dots, p \\ \text{PE } B_m: B_m &\leftarrow B_m + K_1^* A_m & m = 1, 2, \dots, p \end{aligned}$$

The third step involves the shifting of the contents of the upper PEs one place to the left. Thus we have

$$\text{PE } A_m: A_{m-1} \leftarrow A_m \quad m = 2, 3, \dots, p$$

At this point, PE A_1 contains $\gamma_{xx}(2) + K_1 \gamma_{xx}(1)$ while PE B_1 contains $\gamma_{xx}(0) + K_1^* \gamma_{xx}(1)$. Hence the processor A_1 is ready to begin the second cycle by computing

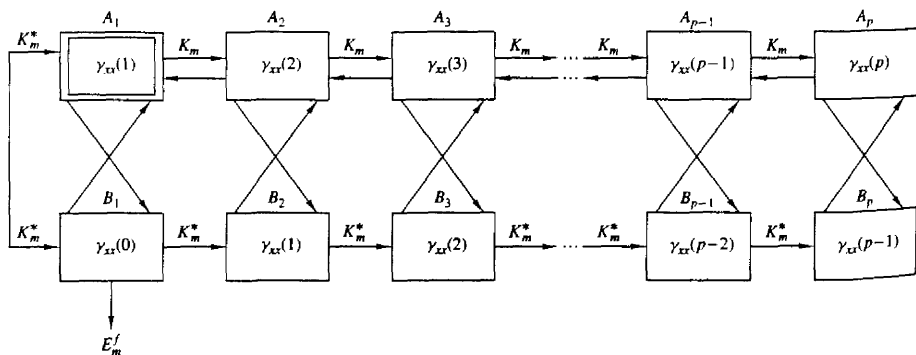


Figure 11.5 Pipelined parallel processor for computing the reflection coefficients

the second reflection coefficient $K_2 = -A_1/B_1$. The three computational steps beginning with the division A_1/B_1 are repeated until all p reflection coefficients are computed. Note that PE B_1 provides the minimum mean-square error E_m^f for each iteration.

If τ_d denotes the time for PE A_1 to perform a (complex) division and τ_{ma} is the time required to perform a (complex) multiplication and an addition, the time required to compute all p reflection coefficients is $p(\tau_d + \tau_{ma})$ for the Schür algorithm.

11.4 PROPERTIES OF THE LINEAR PREDICTION-ERROR FILTERS

Linear prediction filters possess several important properties, which we now describe. We begin by demonstrating that the forward prediction-error filter is minimum phase.

Minimum-phase property of the forward prediction-error filter. We have already demonstrated that the reflection coefficients (K_i) are correlation coefficients, and consequently, $|K_i| \leq 1$ for all i . This condition and the relation $E_m^f = (1 - |K_m|^2)E_{m-1}^f$ can be used to show that the zeros of the prediction-error filter are either all inside the unit circle or they are all on the unit circle.

First, we show that if $E_p^f > 0$, the zeros $|z_i| < 1$ for every i . The proof is by induction. Clearly, for $p = 1$ the system function for the prediction-error filter is

$$A_1(z) = 1 + K_1 z^{-1} \quad (11.4.1)$$

Hence $z_1 = -K_1$ and $E_1^f = (1 - |K_1|^2)E_0^f > 0$. Now, suppose that the hypothesis is true for $p - 1$. Then, if z_i is a root of $A_p(z)$, we have from (11.2.16) and (11.2.18),

$$\begin{aligned} A_p(z_i) &= A_{p-1}(z_i) + K_p z_i^{-1} B_{p-1}(z_i) \\ &= A_{p-1}(z_i) + K_p z_i^{-p} A_{p-1}^*(1/z_i) = 0 \end{aligned} \quad (11.4.2)$$

Hence

$$\frac{1}{K_p} = -\frac{z_i^{-p} A_{p-1}^*(1/z_i)}{A_{p-1}(z_i)} \equiv Q(z_i) \quad (11.4.3)$$

We note that the function $Q(z)$ is all pass. In general, an all-pass function of the form

$$P(z) = \prod_{k=1}^N \frac{z z_k^* + 1}{z + z_k} \quad |z_k| < 1 \quad (11.4.4)$$

satisfies the property that $|P(z)| > 1$ for $|z| < 1$, $|P(z)| = 1$ for $|z| = 1$, and $|P(z)| < 1$ for $|z| > 1$. Since $Q(z) = -P(z)/z$, it follows that $|z_i| < 1$ if $|Q(z_i)| > 1$. Clearly, this is the case since $Q(z_i) = 1/K_p$ and $E_p^f > 0$.

On the other hand, suppose that $E_{p-1}^f > 0$ and $E_p^f = 0$. In this case $|K_p| = 1$ and $|Q(z_i)| = 1$. Since the **MMSE** is zero, the random process $x(n)$ is called **predictable** or **deterministic**. Specifically, a purely sinusoidal random process of the form

$$x(n) = \sum_{k=1}^M \alpha_k e^{j(n\omega_k + \theta_k)} \quad (11.4.5)$$

where the phases $\{\theta_k\}$ are statistically independent and uniformly distributed over $(0, 2\pi)$, has the autocorrelation

$$\gamma_{xx}(m) = \sum_{k=1}^M \alpha_k^2 e^{jm\omega_k} \quad (11.4.6)$$

and the power density spectrum

$$\Gamma_{xx}(f) = \sum_{k=1}^M \alpha_k^2 \delta(f - f_k) \quad f_k = \frac{\omega_k}{2\pi} \quad (11.4.7)$$

This process is predictable with a predictor of order $p \geq M$.

To demonstrate the validity of the statement, consider passing this process through a prediction error filter of order $p \geq M$. The MSE at the output of this filter is

$$\begin{aligned} \mathcal{E}_p^f &= \int_{-1/2}^{1/2} \Gamma_{xx}(f) |A_p(f)|^2 df \\ &= \int_{-1/2}^{1/2} \left[\sum_{k=1}^M \alpha_k^2 \delta(f - f_k) \right] |A_p(f)|^2 df \\ &= \sum_{k=1}^M \alpha_k^2 |A_p(f_k)|^2 \end{aligned} \quad (11.4.8)$$

By choosing M of the p zeros of the **prediction-error** filter to coincide with the frequencies $\{f_k\}$, the **MSE** $\mathcal{E}_p^f$ can be forced to zero. The remaining $p - M$ zeros can be selected arbitrarily to be anywhere inside the unit circle.

Finally, the reader can prove that if a random process consists of a mixture of a continuous power spectral density and a discrete spectrum, the **prediction-error** filter must have **all** its roots inside the unit circle.

Maximum-phase property of the backward prediction-error filter. The system function for the backward prediction error filter of order p is

$$B_p(z) = z^{-p} A_p^*(z^{-1}) \quad (11.4.9)$$

Consequently, the roots of $B_p(z)$ are the reciprocals of the roots of the forward prediction-error filter with system function $A_p(z)$. Hence if $A_p(z)$ is minimum phase, then $B_p(z)$ is maximum phase. However, if the process $x(n)$ is predictable, all the roots of $B_p(z)$ lie on the unit circle.

Whitening properly. Suppose that the random process $x(n)$ is an $AR(p)$ stationary random process that is generated by passing white noise with variance σ_w^2 through an all-pole filter with system function

$$H(z) = \frac{1}{1 + \sum_{k=1}^p a_k z^{-k}} \quad (11.4.10)$$

Then the prediction-error filter of order p has the system function

$$A_p(z) = 1 + \sum_{k=1}^p a_p(k) z^{-k} \quad (11.4.11)$$

where the predictor coefficients $a_p(k) = a_k$. The response of the prediction-error filter is a white noise sequence $\{w(n)\}$. In this case the prediction-error filter whitens the input random process $x(n)$ and is called a whitening filter, as indicated in Section 11.2.

More generally, even if the input process $x(n)$ is not an AR process, the prediction-error filter attempts to remove the correlation among the signal samples of the input process. As the order of the predictor is increased, the predictor output $\hat{x}(n)$ becomes a closer approximation to $x(n)$ and hence the difference $f(n) = \hat{x}(n) - x(n)$ approaches a white noise sequence.

Orthogonality of the backward prediction errors. The backward prediction errors $\{g_m(k)\}$ from different stages in the FIR lattice filter are orthogonal. That is,

$$E[g_m(n)g_l^*(n)] = \begin{cases} 0, & 0 \leq l \leq m-1 \\ E_m^b, & l = m \end{cases} \quad (11.4.12)$$

This property is easily proved by substituting for $g_m(n)$ and $g_l^*(n)$ into (11.4.12) and carrying out the expectation. Thus

$$\begin{aligned} E[g_m(n)g_l^*(n)] &= \sum_{k=0}^m b_m(k) \sum_{j=0}^l b_l^*(j) E[x(n-k)x^*(n-j)] \\ &= \sum_{j=0}^l b_l^*(j) \sum_{k=0}^m b_m(k) \gamma_{xx}(j-k) \end{aligned} \quad (11.4.13)$$

But the normal equations for the backward linear predictor require that

$$\sum_{k=0}^m b_m(k) \gamma_{xx}(j-k) = \begin{cases} 0, & j = 1, 2, \dots, m-1 \\ E_m^b, & j = m \end{cases} \quad (11.4.14)$$

Therefore,

$$E[g_m(n)g_l^*(n)] = \begin{cases} E_m^b = E_m^f, & m = l \\ 0, & 0 \leq l \leq m-1 \end{cases} \quad (11.4.15)$$

Additional properties. There are a number of other interesting properties regarding the forward and backward prediction errors in the FIR lattice filter. These are given here for real-valued data. Their proof is left as an exercise for the reader.

$$(a) E[f_m(n)x(n-i)] = 0, \quad 1 \leq i \leq m$$

$$(b) E[g_m(n)x(n-i)] = 0, \quad 0 \leq i \leq m-1$$

$$(c) E[f_m(n)x(n)] = E[g_m(n)x(n-m)] = E_m$$

$$(d) E[f_i(n)f_j(n)] = E_{\max}(i, j)$$

$$(e) E[f_i(n)f_j(n-t)] = 0, \text{ for } \begin{cases} 1 \leq t \leq i-j, & i > j \\ -1 \geq t \geq i-j, & i < j \end{cases}$$

$$(f) E[g_i(n)g_j(n-t)] = 0, \text{ for } \begin{cases} 0 \leq t \leq i-j, & i > j \\ 0 \geq t \geq i-j+1, & i < j \end{cases}$$

$$(g) E[f_i(n+i)f_j(n+j)] = \begin{cases} E_i, & i = j \\ 0, & i \neq j \end{cases}$$

$$(h) E[g_i(n+i)g_j(n+j)] = E_{\max}(i, j)$$

$$(i) E[f_i(n)g_j(n)] = \begin{cases} K_j E_i, & i \geq j, \\ 0, & i < j \end{cases} \quad i, j \geq 0, \quad K_0 = 1$$

$$(j) E[f_i(n)g_i(n-1)] = -K_{i+1} E_i$$

$$(k) E[g_i(n-1)x(n)] = E[f_i(n+1)x(n-1)] = -K_{i+1} E_i$$

$$(l) E[f_i(n)g_j(n-1)] = \begin{cases} 0, & i > j \\ -K_{j+1} E_i, & i \leq j \end{cases}$$

11.5 AR LATTICE AND ARMA LATTICE-LADDER FILTERS

In Section 11.2 we showed the relationship of the all-zero FIR lattice to linear prediction. The linear predictor with transfer function,

$$A_p(z) = 1 + \sum_{k=1}^p a_p(k)z^{-k} \quad (11.5.1)$$

when excited by an input random process $\{x(n)\}$, produces an output that **approaches** a white noise sequence as $p \rightarrow \infty$. On the other hand, if the **input** process is an **AR(p)**, the output of $A_p(z)$ is white. **Since** $A_p(z)$ generates a **MA(p)** process when excited with a white noise sequence, the all-zero lattice is sometimes **called** a MA lattice.

In the following section, we develop the lattice structure for the inverse **filter** $1/A_p(z)$, called the **AR** lattice, and the lattice-ladder structure for an **ARMA** process.

11.5.1 AR Lattice Structure

Let us consider an all-pole system with system function

$$H(z) = \frac{1}{1 + \sum_{k=1}^p a_p(k)z^{-k}} \quad (11.5.2)$$

The difference equation for this *IIR* system is

$$y(n) = - \sum_{k=1}^p a_p(k)y(n-k) + x(n) \quad (11.5.3)$$

Now suppose that we interchange the roles of the input and output [i.e., interchange $x(n)$ with $y(n)$ in (11.5.3)] obtaining the difference equation

$$x(n) = - \sum_{k=1}^p a_p(k)x(n-k) + y(n)$$

or, equivalently,

$$y(n) = x(n) + \sum_{k=1}^p a_p(k)x(n-k) \quad (11.5.4)$$

We observe that (11.5.4) is a difference equation for an *FIR* system with system function $A_p(z)$. Thus an all-pole *IIR* system can be converted to an all-zero system by interchanging the roles of the input and output.

Based on this observation, we can obtain the structure of an $\text{AR}(p)$ lattice from a $\text{MA}(p)$ lattice by interchanging the input with the output. Since the $\text{MA}(p)$ lattice has $y(n) = f_p(n)$ as its output and $x(n) = f_0(n)$ is the input, we let

$$\begin{aligned} x(n) &= f_p(n) \\ y(n) &= f_0(n) \end{aligned} \quad (11.5.5)$$

These definitions dictate that the quantities $\{f_m(n)\}$ be computed in descending order. This computation can be accomplished by **rearranging** the recursive equation for $\{f_m(n)\}$ in (11.2.4) and solving for $f_{m-1}(n)$ in **terms** of $f_m(n)$. Thus **we** obtain

$$f_{m-1}(n) = f_m(n) - K_m g_{m-1}(n-1) \quad m = p, p-1, \dots, 1$$

The equation for $g_m(n)$ remains unchanged. The result of these changes is the set of equations

$$\begin{aligned} x(n) &= f_p(n) \\ f_{m-1}(n) &= f_m(n) - K_m g_{m-1}(n-1) \\ g_m(n) &= K_m^* f_{m-1}(n) + g_{m-1}(n-1) \\ y(n) &= f_0(n) = g_0(n) \end{aligned} \quad (11.5.6)$$

The **corresponding** structure for the $\text{AR}(p)$ lattice is shown in Fig. 11.6. Note that the all-pole lattice structure has an all-zero path with input $g_0(n)$ and output $g_p(n)$,

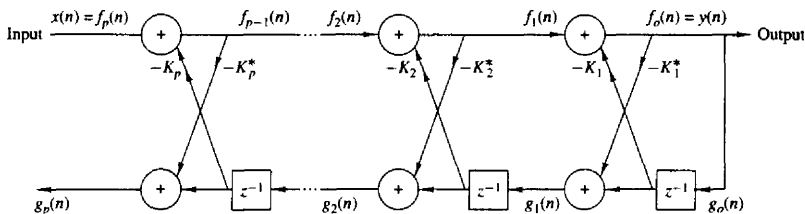


Figure 11.6 Lattice structure for an all-pole system

which is identical to the all-zero path in the **Ma**(p) lattice structure. This is not surprising, since the equation for $g_m(n)$ is identical in the two lattice structures.

We also observe that the **AR**(p) and **MA**(p) lattice structures are characterized by the same parameters, namely, the reflection coefficients $\{K_i\}$. Consequently, the equations given in (11.2.21) and (11.2.23) for converting between the system parameters $\{a_p(k)\}$ in the direct-form realizations of the all-zero system $A_p(z)$ and the lattice parameters $\{K_i\}$ of the **MA**(p) lattice structure, apply as well to the all-pole structures.

11.5.2 ARMA Processes and Lattice-Ladder Filters

The all-pole lattice provides the basic building block for lattice-type structures that implement IIR systems that contain both poles and zeros. To construct the appropriate structure, let us consider an **IIR** system with system function

$$H(z) = \frac{\sum_{k=0}^q c_q(k)z^{-k}}{1 + \sum_{k=1}^p a_p(k)z^{-k}} = \frac{C_q(z)}{A_p(z)} \quad (11.5.7)$$

Without loss of generality, we assume that $p \geq q$.

This system is described by the difference equations

$$\begin{aligned} v(n) &= -\sum_{k=1}^p a_p(k)v(n-k) + x(n) \\ y(n) &= \sum_{k=0}^q c_q(k)v(n-k) \end{aligned} \quad (11.5.8)$$

obtained by viewing the system as a cascade of an all-pole system followed by an all-zero system. From (11.5.8) we observe that the output $y(n)$ is simply a linear combination of delayed outputs from the all-pole system.

Since zeros result from forming a linear combination of previous outputs, we can carry over this observation to construct a pole-zero system using the all-pole

lattice structure as the basic building block. We have clearly observed that $g_m(n)$ in the all-pole lattice can be expressed as a linear combination of present and past outputs. In fact, the system

$$H_b(z) \equiv \frac{G_m(z)}{Y(z)} = B_m(z) \quad (11.5.9)$$

is an all-zero system. Therefore, any linear combination of $\{g_m(n)\}$ is also an all-zero filter.

Let us begin with an all-pole lattice filter with coefficients K_m , $1 \leq m \leq p$, and add a **ladder** part by taking as the output, a weighted linear combination of $\{g_m(n)\}$. The result is a pole-zero filter that has the **lattice-ladder** structure shown in Fig. 11.7. Its output is

$$y(n) = \sum_{k=0}^q \beta_k g_k(n) \quad (11.5.10)$$

where $\{\beta_k\}$ are the parameters that determine the zeros of the system. The system function corresponding to (11.5.10) is

$$H(z) = \frac{Y(z)}{X(z)} = \sum_{k=0}^q \beta_k \frac{G_k(z)}{X(z)} \quad (11.5.11)$$

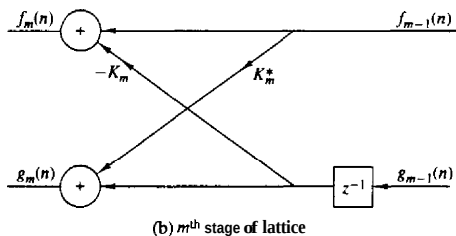
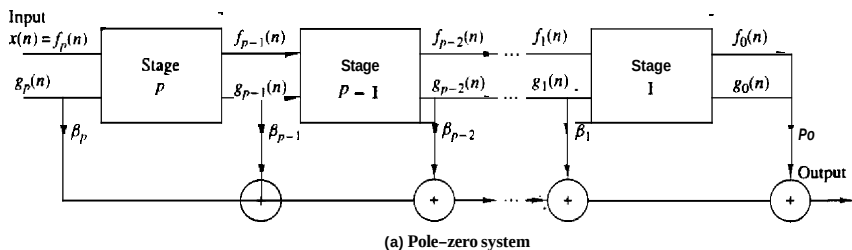


Figure 11.7 Lattice-ladder structure for pole-zero system

Since $X(z) = F_p(z)$ and $F_0(z) = G_0(z)$, (11.5.11), can be expressed as

$$\begin{aligned} H(z) &= \sum_{k=0}^q \beta_k \frac{G_k(z)}{G_0(z)} \frac{F_0(z)}{F_p(z)} \\ &= \frac{1}{A_p(z)} \sum_{k=0}^q \beta_k B_k(z) \end{aligned} \quad (11.5.12)$$

Therefore,

$$C_q(z) = \sum_{k=0}^q \beta_k B_k(z) \quad (11.5.13)$$

This is the desired relationship that can be used to determine the weighting coefficients $\{\beta_k\}$ as previously shown in Section 7.3.5.

Given the polynomials $C_q(z)$ and $A_p(z)$, where $p \geq q$, the reflection coefficients $\{K_i\}$ are determined first from the coefficients $\{a_p(k)\}$. By means of the step-down recursive relation given by (11.2.22), we also obtain the polynomials $B_k(z)$, $k = 1, 2, \dots, p$. Then the ladder parameters can be obtained from (11.5.13), which can be expressed as

$$\begin{aligned} C_m(z) &= \sum_{k=0}^{m-1} \beta_k B_k(z) + \beta_m B_m(z) \\ &= C_{m-1}(z) + \beta_m B_m(z) \end{aligned} \quad (11.5.14)$$

or, equivalently,

$$C_{m-1}(z) = C_m(z) - \beta_m B_m(z) \quad m = p, p-1, \dots, 1 \quad (11.5.15)$$

By running this recursive relation backward, we can generate all the lower-degree polynomials, $C_m(z)$, $m = p-1, \dots, 1$. Since $b_m(m) = 1$, the parameters β_m are determined from (11.5.15) by setting

$$\beta_m = c_m(m) \quad m = p, p-1, \dots, 1, 0$$

When excited by a white noise sequence, this lattice-ladder filter structure generates an ARMA(p, q) process that has a power density spectrum

$$\Gamma_{xx}(f) = \sigma_w^2 \frac{|C_q(f)|^2}{|A_p(f)|^2} \quad (11.5.16)$$

and an autocorrelation function that satisfies (11.1.18), where σ_w^2 is the variance of the input white noise sequence.

11.6 WIENER FILTERS FOR FILTERING AND PREDICTION

In many practical applications we are given an input signal $\{x(n)\}$, consisting of the sum of a desired signal $\{s(n)\}$ and an undesired noise or interference $\{w(n)\}$, and we are asked to design a **filter** that suppresses the undesired interference

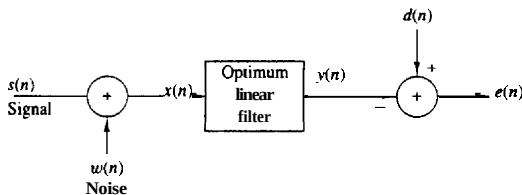


Figure 11.8 Model for linear estimation problem

component. In such a case, the objective is to design a system that filters out the additive interference while preserving the characteristics of the desired signal $\{s(n)\}$.

In this section we treat the problem of signal estimation in the presence of an additive noise disturbance. The estimator is constrained to be a linear filter with impulse response $\{h(n)\}$, designed so that its output approximates some specified desired signal sequence $\{d(n)\}$. Figure 11.8 illustrates the linear estimation problem.

The input sequence to the filter is $x(n) = s(n) + w(n)$, and its output sequence is $y(n)$. The difference between the desired signal and the filter output is the error sequence $e(n) = d(n) - y(n)$.

We distinguish three special cases:

1. If $d(n) = s(n)$, the linear estimation problem is referred to as **filtering**.
2. If $d(n) = s(n + D)$, where $D > 0$, the linear estimation problem is referred to as signal **prediction**. Note that this problem is different than the prediction considered earlier in this chapter, where $d(n) = x(n + D)$, $D \geq 0$.
3. If $d(n) = s(n - D)$, where $D > 0$, the linear estimation problem is referred to as signal **smoothing**.

Our treatment will concentrate on filtering and prediction.

The criterion selected for optimizing the filter impulse response $\{h(n)\}$ is the minimization of the mean-square error. This criterion has the advantages of simplicity and mathematical tractability.

The basic assumptions are that the sequences $\{s(n)\}$, $\{w(n)\}$, and $\{d(n)\}$ are zero mean and wide-sense stationary. The linear filter will be assumed to be either **FIR** or **IIR**. If it is **IIR**, we assume that the input data $\{x(n)\}$ are available over the infinite past. We begin with the design of the optimum **FIR** filter. The optimum linear filter, in the sense of minimum mean-square error (**MMSE**), is called a **Wiener filter**.

11.6.1 FIR Wiener Filter

Suppose that the filter is constrained to be of length M with coefficients $\{h_k, 0 \leq k \leq M - 1\}$. Hence its output $y(n)$ depends on the finite data record $x(n)$,

$$x(n-1), \dots, x(n-M+1),$$

$$y(n) = \sum_{k=0}^{M-1} h(k)x(n-k) \quad (11.6.1)$$

The mean-square value of the error between the desired output $d(n)$ and $y(n)$ is

$$\begin{aligned} \mathcal{E}_M &= E|e(n)|^2 \\ &= E \left| d(n) - \sum_{k=0}^{M-1} h(k)x(n-k) \right|^2 \end{aligned} \quad (11.6.2)$$

Since this is a quadratic function of the filter coefficients, the minimization of $\mathcal{E}_M$ yields the set of linear equations

$$\sum_{k=0}^{M-1} h(k)\gamma_{xx}(l-k) = \gamma_{dx}(l) \quad l = 0, 1, \dots, M-1 \quad (11.6.3)$$

where $\gamma_{xx}(k)$ is the autocorrelation of the input sequence $\{x(n)\}$ and $\gamma_{dx}(k) = E[d(n)x^*(n-k)]$ is the crosscorrelation between the desired sequence $\{d(n)\}$ and the input sequence $\{x(n)\}$, $0 \leq n \leq M-1$. The set of linear equations that specify the optimum filter is called the **Wiener-Hopf equation**. These equations are also called the **normal equations**, encountered earlier in the chapter in the context of linear one-step prediction.

In general, the equations in (11.6.3) can be expressed in matrix form as

$$\mathbf{\Gamma}_M \mathbf{h}_M = \boldsymbol{\gamma}_d \quad (11.6.4)$$

where $\mathbf{\Gamma}_M$ is an $M \times M$ (**Hermitian**) Toeplitz matrix with elements $\Gamma_{lk} = \gamma_{xx}(l-k)$ and $\boldsymbol{\gamma}_d$ is the $M \times 1$ crosscorrelation vector with elements $\gamma_{dx}(l)$, $l = 0, 1, \dots, M-1$. The solution for the optimum filter coefficients is

$$\mathbf{h}_{\text{opt}} = \mathbf{\Gamma}_M^{-1} \boldsymbol{\gamma}_d \quad (11.6.5)$$

and the resulting minimum MSE achieved by the Wiener filter is

$$\text{MMSE}_M = \min_{\mathbf{h}_M} \mathcal{E}_M = \sigma_d^2 - \sum_{k=0}^{M-1} h_{\text{opt}}(k) \gamma_{dx}^*(k) \quad (11.6.6)$$

or, equivalently,

$$\text{MMSE}_M = \sigma_d^2 - \boldsymbol{\gamma}_d^* \mathbf{\Gamma}_M^{-1} \boldsymbol{\gamma}_d \quad (11.6.7)$$

where $\sigma_d^2 = E|d(n)|^2$.

Let us consider some special cases of (11.6.3). If we are dealing with filtering, the $d(n) = s(n)$. Furthermore, if $s(n)$ and $w(n)$ are uncorrelated random **sequences**, as is usually the case in practice, then

$$\begin{aligned} \gamma_{xx}(k) &= \gamma_{ss}(k) + \gamma_{ww}(k) \\ \gamma_{dx}(k) &= \gamma_{ss}(k) \end{aligned} \quad (11.6.8)$$

and the normal equations in (11.6.3) become

$$\sum_{k=0}^{M-1} h(k)[\gamma_{ss}(l-k) + \gamma_{ww}(l-k)] = \gamma_{ss}(l) \quad l = 0, 1, \dots, M-1 \quad (11.6.9)$$

If we are dealing with prediction, then $d(n) = s(n+D)$ where $D > 0$. Assuming that $s(n)$ and $w(n)$ are uncorrelated random sequences, we have

$$\gamma_{ds}(k) = \gamma_{ss}(l+D) \quad (11.6.10)$$

Hence the equations for the Wiener prediction filter become

$$\sum_{k=0}^{M-1} h(k)[\gamma_{ss}(l-k) + \gamma_{ww}(l-k)] = \gamma_{ss}(l+D) \quad l = 0, 1, \dots, M-1 \quad (11.6.11)$$

In all these cases, the correlation matrix to be inverted is **Toeplitz**. Hence the (generalized) Levinson–Durbin algorithm may be used to solve for the optimum filter coefficients.

Example 11.6.1

Let us consider a signal $x(n) = s(n) + w(n)$, where $s(n)$ is an **AR(1)** process that satisfies the difference equation

$$s(n) = 0.6s(n-1) + v(n)$$

where $\{v(n)\}$ is a white noise sequence with variance $\sigma_v^2 = 0.64$, and $\{w(n)\}$ is a white noise sequence with variance $\sigma_w^2 = 1$. We will design a Wiener filter of length $M = 2$ to estimate $\{s(n)\}$.

Solution Since $\{s(n)\}$ is obtained by exciting a single-pole filter by white noise, the power spectral density of $s(n)$ is

$$\begin{aligned} \Gamma_{ss}(f) &= \sigma_v^2 |H(f)|^2 \\ &= \frac{0.64}{|1 - 0.6e^{-j2\pi f}|^2} \\ &= \frac{0.64}{1.36 - 1.2\cos 2\pi f} \end{aligned}$$

The corresponding **autocorrelation** sequence $\{\gamma_{ss}(m)\}$ is

$$\gamma_{ss}(m) = (0.6)^{|m|}$$

The equations for the filter coefficients are

$$\begin{aligned} 2h(0) + 0.6h(1) &= 1 \\ 0.6h(0) + 2h(1) &= 0.6 \end{aligned}$$

Solution of these equations yields the result

$$h(0) = 0.451 \quad h(1) = 0.165$$